

Deep Learning Final Project

Deep Learning Regression for Stellar Age Estimation of Red Giant Stars

Problem and Dataset

Problem: Estimating the age of red giant stars from their observable properties using a deep learning regression model. Stellar ages are notoriously difficult to measure directly — they are typically inferred through asteroseismology (analysis of stellar oscillations) combined with spectroscopy. Training a neural network to predict stellar ages from readily observable parameters could enable rapid age estimation for large stellar surveys.

Dataset: The **APO-K2 catalogue** (Schonhut-Stasik et al. 2024, AJ, 167, 208), accessed via the VizieR astronomical database ([J/AJ/167/208/table3](https://vizier.cfa.harvard.edu/J/AJ/167/208/table3)). This catalogue combines:

- **APOGEE spectroscopic parameters:** effective temperature (T_{eff}), surface gravity ($\log g$), metallicity ($[\text{Fe}/\text{H}]$), alpha-element abundance ($[\alpha/\text{M}]$), oxygen abundance ($[\text{O}/\text{Fe}]$).
- **Asteroseismic parameters** from K2: frequency of maximum oscillation power (ν_{max}), large frequency spacing ($\Delta \nu$).
- **Derived stellar properties:** mass (M_{star}), radius (R_{star}).
- **Target variable:** Stellar age (mode of the posterior distribution, Age-mode), in Gyr.

The raw dataset contains 1912 red giant stars. After quality filtering (removing non-physical values, high-uncertainty ages, and missing data), 936 stars remain for modelling.

```
In [38]: import os
os.makedirs('plots', exist_ok=True)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Astronomy data access
from astroquery.vizier import Vizier

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Input
```

```
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

np.random.seed(42)
tf.random.set_seed(42)
```

```
In [39]: # Loading the APO-K2 stellar age catalog from VizieR
catalog_id = "J/AJ/167/208/table3"

vizier = Vizier(
    columns=[
        "EPIC", "GaiaEDR3", "Teff", "e_Teff", "logg", "e_logg",
        "[Fe/H]", "e_[Fe/H]", "[a/M]", "e_[a/M]", "[O/Fe]", "e_[O/Fe]",
        "Mstar", "e_Mstar", "Rstar", "e_Rstar", "logg-seis", "e_logg-seis",
        "numax", "e_numax", "deltanu", "e_deltanu",
        "Age", "e_Age", "E_Age", "Age-mode"
    ],
    row_limit=-1
)

tables = vizier.get_catalogs(catalog_id)
table = tables[0]
df = table.to_pandas()

print("Shape:", df.shape)
print(df.head())
print(df.columns.tolist())
```

Shape: (1912, 26)

	EPIC	GaiaEDR3	Teff	e_Teff	logg	e_logg	[Fe/H]	\
0	211837778	656372815030872192	4759	9	2.38	0.02	-0.306	
1	210915682	58475709154017280	4757	9	2.44	0.03	-0.335	
2	210352614	37590412768055040	4755	9	2.36	0.03	-0.556	
3	214710068	4073182748305313536	4643	10	2.18	0.03	-0.774	
4	212351370	3607531430651890048	4999	11	2.47	0.03	-0.461	

	e_[Fe/H]	[a/M]	e_[a/M]	...	logg-seis	e_logg-seis	numax	e_numax
0	0.008	0.063	0.006	...	2.45	0.01	35.000000	0.9
1	0.008	0.077	0.007	...	2.48	0.01	37.700001	0.5
2	0.008	0.123	0.008	...	2.44	0.01	34.400002	0.7
3	0.010	0.301	0.010	...	2.18	0.01	19.100000	0.5
4	0.008	0.104	0.007	...	2.55	0.01	42.599998	0.7

	deltanu	e_deltanu	Age	e_Age	E_Age	Age-mode
0	3.97	0.01	2.3	0.5	0.700000	2.2
1	4.37	0.12	3.9	1.9	3.800000	3.5
2	4.38	0.12	9.2	4.6	9.200000	8.4
3	2.91	0.14	15.4	11.4	42.900002	12.2
4	4.80	0.07	3.0	1.0	1.400000	2.8

[5 rows x 26 columns]

```
['EPIC', 'GaiaEDR3', 'Teff', 'e_Teff', 'logg', 'e_logg', '[Fe/H]', 'e_[Fe/H]', '[a/M]', 'e_[a/M]', '[O/Fe]', 'e_[O/Fe]', 'Mstar', 'e_Mstar', 'Rstar', 'e_Rstar', 'logg-seis', 'e_logg-seis', 'numax', 'e_numax', 'deltanu', 'e_deltanu', 'Age', 'e_Age', 'E_Age', 'Age-mode']
```

In [40]: *# Cleaning column names and inspecting the missing values*

```
df = df.rename(columns={
    "Teff": "teff",
    "e_Teff": "teff_err",
    "logg": "logg_spec",
    "e_logg": "logg_spec_err",
    "[Fe/H]": "feh",
    "e_[Fe/H]": "feh_err",
    "[a/M]": "alpha_m",
    "e_[a/M]": "alpha_m_err",
    "[O/Fe]": "o_fe",
    "e_[O/Fe]": "o_fe_err",
    "Mstar": "mass",
    "e_Mstar": "mass_err",
    "Rstar": "radius",
    "e_Rstar": "radius_err",
    "logg-seis": "logg_seis",
    "e_logg-seis": "logg_seis_err",
    "Age": "age_median",
    "e_Age": "age_err_low",
    "E_Age": "age_err_high",
    "Age-mode": "age_mode"
})

print(df.info())
print("\nMissing values:\n", df.isnull().sum().sort_values(ascending=False))
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1912 entries, 0 to 1911
Data columns (total 26 columns):
#   Column                Non-Null Count  Dtype
---  -
0   EPIC                   1912 non-null   int32
1   GaiaEDR3               1912 non-null   int64
2   teff                   1912 non-null   int16
3   teff_err               1912 non-null   int16
4   logg_spec              1912 non-null   float32
5   logg_spec_err          1912 non-null   float32
6   feh                    1912 non-null   float32
7   feh_err                1912 non-null   float32
8   alpha_m                1912 non-null   float32
9   alpha_m_err            1912 non-null   float32
10  o_fe                   1912 non-null   float32
11  o_fe_err               1912 non-null   float32
12  mass                   1912 non-null   float32
13  mass_err               1912 non-null   float32
14  radius                 1912 non-null   float32
15  radius_err             1912 non-null   float32
16  logg_seis              1912 non-null   float32
17  logg_seis_err          1912 non-null   float32
18  numax                  1912 non-null   float32
19  e_numax                1912 non-null   float32
20  deltanu                1912 non-null   float32
21  e_deltanu              1912 non-null   float32
22  age_median             1912 non-null   float32
23  age_err_low            1912 non-null   float32
24  age_err_high           1912 non-null   float32
25  age_mode                1912 non-null   float32
dtypes: float32(22), int16(2), int32(1), int64(1)
memory usage: 194.3 KB
None

```

```

Missing values:
EPIC                0
GaiaEDR3            0
age_err_high        0
age_err_low         0
age_median          0
e_deltanu           0
deltanu             0
e_numax             0
numax               0
logg_seis_err       0
logg_seis           0
radius_err          0
radius              0
mass_err            0
mass                0
o_fe_err            0
o_fe                0
alpha_m_err         0
alpha_m             0
feh_err             0

```

```
feh            0
logg_spec_err  0
logg_spec      0
teff_err       0
teff           0
age_mode       0
dtype: int64
```

In [41]: *# Defining Target*

```
# Selected input features
feature_cols = [
    "teff",
    "logg_spec",
    "feh",
    "alpha_m",
    "o_fe",
    "mass",
    "radius",
    "logg_seis",
    "numax",
    "deltanu"
]

target_col = "age_mode"
error_cols = [
    "teff_err", "logg_spec_err", "feh_err", "alpha_m_err", "o_fe_err",
    "mass_err", "radius_err", "logg_seis_err", "e_numax", "e_deltanu",
    "age_err_low", "age_err_high"
]

use_cols = feature_cols + [target_col] + error_cols

data = df[use_cols].copy()
print(data.head())
```

	teff	logg_spec	feh	alpha_m	o_fe	mass	radius	logg_seis	numa
x \									
0	4759	2.38	-0.306	0.063	0.105	1.49	12.00	2.45	35.00000
0									
1	4757	2.44	-0.335	0.077	0.159	1.26	10.65	2.48	37.70000
1									
2	4755	2.36	-0.556	0.123	0.259	0.95	9.68	2.44	34.40000
2									
3	4643	2.18	-0.774	0.301	0.369	0.81	12.05	2.18	19.10000
0									
4	4999	2.47	-0.461	0.104	0.201	1.34	10.20	2.55	42.59999
8									

	deltanu	...	feh_err	alpha_m_err	o_fe_err	mass_err	radius_err	\
0	3.97	...	0.008	0.006	0.018	0.12	0.31	
1	4.37	...	0.008	0.007	0.019	0.23	0.61	
2	4.38	...	0.008	0.008	0.020	0.17	0.56	
3	2.91	...	0.010	0.010	0.023	0.25	1.17	
4	4.80	...	0.008	0.007	0.027	0.15	0.35	

	logg_seis_err	e_numax	e_deltanu	age_err_low	age_err_high
0	0.01	0.9	0.01	0.5	0.700000
1	0.01	0.5	0.12	1.9	3.800000
2	0.01	0.7	0.12	4.6	9.200000
3	0.01	0.5	0.14	11.4	42.900002
4	0.01	0.7	0.07	1.0	1.400000

[5 rows x 23 columns]

```
In [42]: # Data cleaning and physically sensible filtering

data = data.dropna(subset=feature_cols + [target_col]).copy()

# Removing obviously non-physical or extreme target values

data = data[(data["age_mode"] > 0.0) & (data["age_mode"] <= 20.0)]

# Removing rows with invalid seismic quantities
data = data[(data["numax"] > 0) & (data["deltanu"] > 0)]

# Removing stars with very large age uncertainty if available
if "age_err_low" in data.columns and "age_err_high" in data.columns:
    data = data[(data["age_err_low"] < 5.0) & (data["age_err_high"] < 5.0)]

print("Cleaned shape:", data.shape)
print(data.describe())
```

Cleaned shape: (936, 23)

	teff	logg_spec	feh	alpha_m	o_fe	\
count	936.000000	936.000000	936.000000	936.000000	936.000000	
mean	4701.530983	2.369338	-0.304338	0.113724	0.164438	
std	204.546027	0.248243	0.259624	0.095616	0.140254	
min	4175.000000	1.520000	-0.999000	0.000000	-0.531000	
25%	4554.750000	2.210000	-0.479000	0.035000	0.057750	
50%	4695.500000	2.380000	-0.301000	0.082000	0.133000	
75%	4842.250000	2.520000	-0.127000	0.197250	0.264250	
max	5374.000000	3.270000	0.505000	0.387000	0.813000	
	mass	radius	logg_seis	numax	deltanu	...
count	936.000000	936.000000	936.000000	936.000000	936.000000	...
mean	1.331389	12.411443	2.393066	35.397011	4.001624	...
std	0.332136	3.538416	0.235654	20.814911	1.623964	...
min	0.780000	5.350000	1.680000	6.100000	1.000000	...
25%	1.080000	10.207500	2.250000	22.675001	2.980000	...
50%	1.260000	11.365000	2.420000	32.950001	3.975000	...
75%	1.520000	13.800000	2.510000	39.700001	4.590000	...
max	2.590000	31.410000	3.120000	155.800003	12.740000	...
	feh_err	alpha_m_err	o_fe_err	mass_err	radius_err	\
count	936.000000	936.000000	936.000000	936.000000	936.000000	
mean	0.009581	0.008764	0.022928	0.147468	0.449049	
std	0.003241	0.011295	0.013589	0.095551	0.322046	
min	0.006000	0.004000	0.008000	0.020000	0.050000	
25%	0.008000	0.006000	0.015000	0.090000	0.257500	
50%	0.008000	0.007000	0.019000	0.120000	0.370000	
75%	0.011000	0.009000	0.026000	0.180000	0.550000	
max	0.035000	0.284000	0.129000	0.920000	3.120000	
	logg_seis_err	e_numax	e_deltanu	age_err_low	age_err_high	
count	936.000000	936.000000	936.000000	936.000000	936.000000	
mean	0.008632	0.633333	0.049316	1.443056	2.183547	
std	0.005891	0.392469	0.035735	0.884891	1.371750	
min	0.000000	0.000000	0.000000	0.100000	0.100000	
25%	0.010000	0.300000	0.020000	0.700000	1.000000	
50%	0.010000	0.600000	0.040000	1.300000	2.000000	
75%	0.010000	0.800000	0.070000	2.100000	3.300000	
max	0.040000	2.000000	0.290000	3.800000	4.900000	

[8 rows x 23 columns]

Why Preprocessing Was Necessary

Several preprocessing steps were applied and are essential for this task:

1. **Column renaming** (Cell 3): The original VizieR catalog uses specialised astronomical column names (e.g., `[Fe/H]` , `e_[a/M]`). These are renamed to clean, code-friendly identifiers for easier manipulation.
2. **Missing value removal**: Any rows with NaN values in the feature or target columns are dropped to ensure the model trains on complete data.

3. Physical filtering:

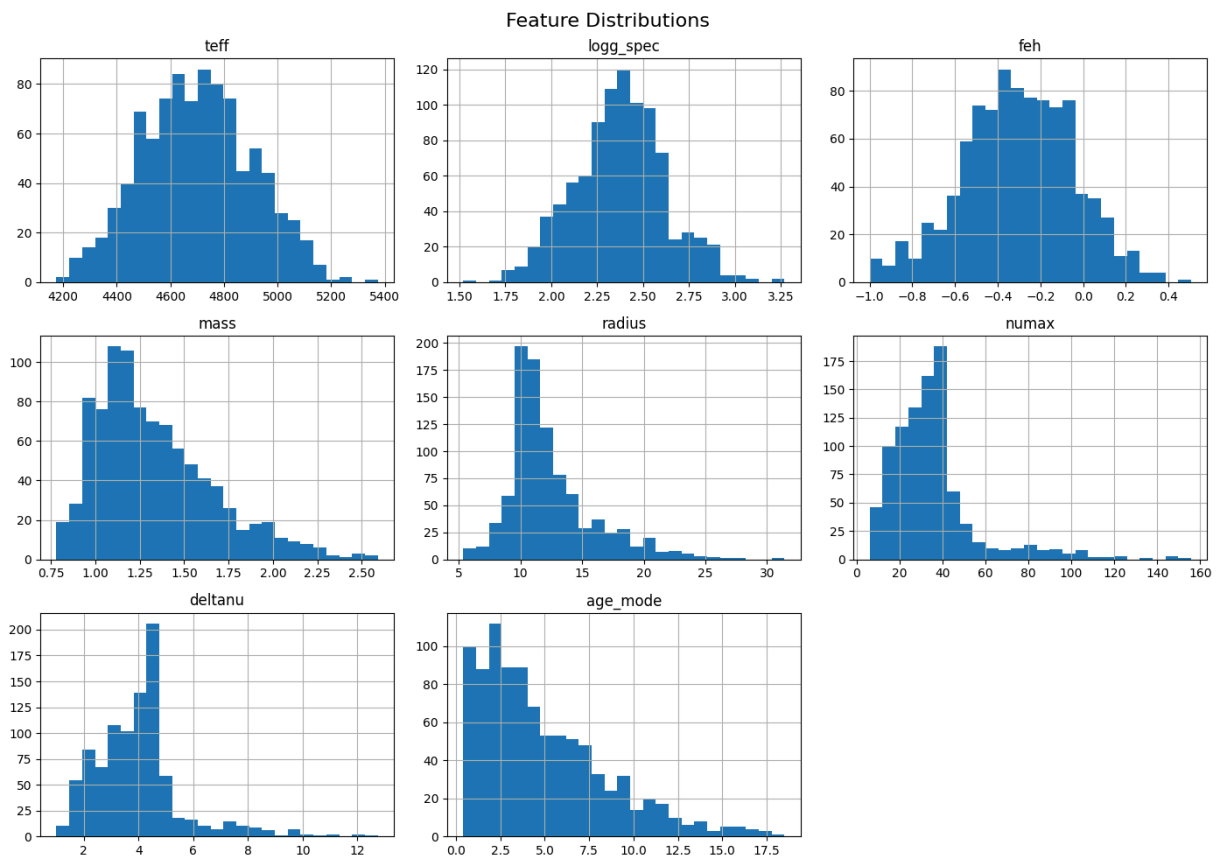
- Ages are restricted to 0–20 Gyr (the approximate age of the universe) to remove non-physical or extreme outlier values.
- Seismic quantities (`numax` , `deltanu`) must be positive.
- Rows with large age uncertainties (`age_err_low > 5` or `age_err_high > 5`) are excluded, as highly uncertain label values would degrade training.

4. **StandardScaler normalisation** (Cell 10): Neural networks perform best when input features are on similar scales. StandardScaler centres each feature to mean=0 and std=1, which:

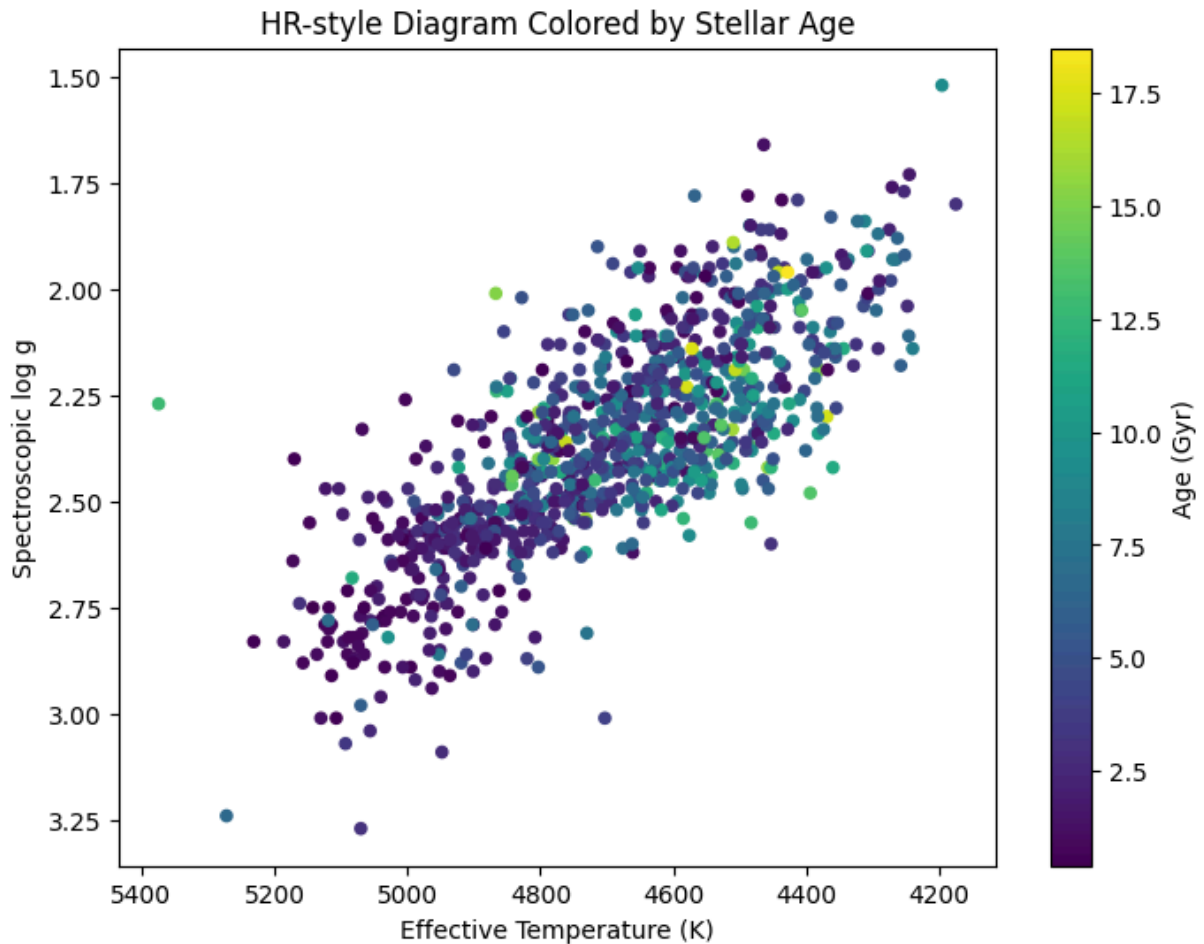
- Prevents features with large numerical ranges (e.g., `teff` ~4000–5400 K) from dominating over smaller-scale features (e.g., `logg` ~1.5–3.3).
- Helps gradient-based optimisation converge faster and more reliably.

After cleaning, the dataset is reduced from 1912 to 936 high-quality samples.

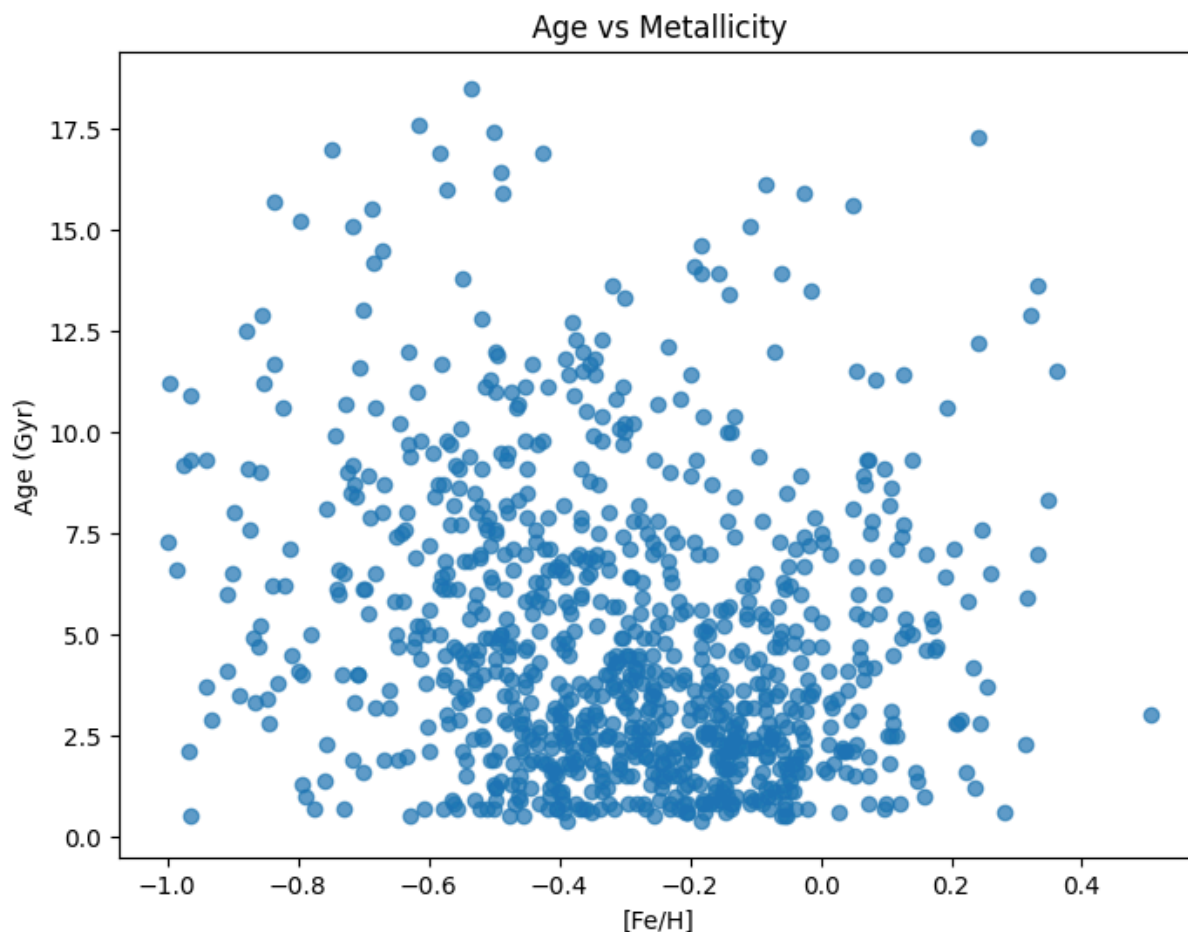
```
In [43]: # Histograms of important features
data[["teff", "logg_spec", "feh", "mass", "radius", "numax", "deltanu", "age_mode"]]
figsize=(14, 10), bins=25
)
plt.suptitle("Feature Distributions", fontsize=16)
plt.tight_layout()
plt.savefig('plots/feature_distributions.png', dpi=150, bbox_inches='tight')
plt.show()
```



```
In [44]: # HR-style diagram: Teff vs logg (common stellar diagnostic view)
plt.figure(figsize=(8, 6))
plt.scatter(data["teff"], data["logg_spec"], c=data["age_mode"], s=20)
plt.gca().invert_xaxis()
plt.gca().invert_yaxis()
plt.xlabel("Effective Temperature (K)")
plt.ylabel("Spectroscopic log g")
plt.title("HR-style Diagram Colored by Stellar Age")
plt.colorbar(label="Age (Gyr)")
plt.savefig('plots/hr_style_diagram_colored_by_stellar_age.png', dpi=150, bb
plt.show()
```



```
In [45]: # Age vs metallicity
plt.figure(figsize=(8, 6))
plt.scatter(data["feh"], data["age_mode"], alpha=0.7)
plt.xlabel("[Fe/H]")
plt.ylabel("Age (Gyr)")
plt.title("Age vs Metallicity")
plt.savefig('plots/age_vs_metallicity.png', dpi=150, bbox_inches='tight')
plt.show()
```



```
In [46]: X = data[feature_cols].copy()
y = data[target_col].copy()

# Train/temp split
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42
)

# Validation/test split
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42
)

print("Train shape:", X_train.shape)
print("Validation shape:", X_val.shape)
print("Test shape:", X_test.shape)
```

```
Train shape: (655, 10)
Validation shape: (140, 10)
Test shape: (141, 10)
```

```
In [47]: scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

Network Architecture Choice

A fully-connected (dense) feedforward neural network is chosen for this regression task. This architecture is well-suited because:

1. **Tabular data:** The input features are tabular numerical values (stellar parameters), not images or sequences, so convolutional or recurrent architectures are unnecessary.
2. **Non-linear relationships:** Dense layers with ReLU activation can model complex non-linear mappings between stellar observables and age.
3. **Moderate dataset size:** With ~936 samples after cleaning, a moderately-sized network (128 → 64 → 32 neurons) provides enough capacity without excessive risk of overfitting.

Layer-by-layer design:

- **Input layer** (shape = 10): Accepts the 10 normalised stellar features.
- **Dense(128, ReLU) + Dropout(0.2)**: First hidden layer extracts broad feature patterns. Dropout regularisation prevents co-adaptation of neurons.
- **Dense(64, ReLU) + Dropout(0.2)**: Second hidden layer refines the representations.
- **Dense(32, ReLU)**: Third hidden layer provides final abstraction before the output.
- **Dense(1, linear)**: Single output neuron with linear activation for regression — predicts the continuous stellar age in Gyr.

Optimiser: Adam with learning rate 1e-3 — adaptive learning rate optimiser that works well with sparse gradients and noisy data.

Loss: Mean Squared Error (MSE) — standard for regression tasks.

```
In [48]: baseline_model = Sequential([
    Input(shape=(X_train_scaled.shape[1],)),
    Dense(128, activation="relu"),
    Dropout(0.2),
    Dense(64, activation="relu"),
    Dropout(0.2),
    Dense(32, activation="relu"),
    Dense(1, activation="linear")
])

baseline_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="mse",
    metrics=["mae"]
)

baseline_model.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Par
dense_16 (Dense)	(None, 128)	1
dropout_11 (Dropout)	(None, 128)	
dense_17 (Dense)	(None, 64)	8
dropout_12 (Dropout)	(None, 64)	
dense_18 (Dense)	(None, 32)	2
dense_19 (Dense)	(None, 1)	

Total params: 11,777 (46.00 KB)

Trainable params: 11,777 (46.00 KB)

Non-trainable params: 0 (0.00 B)

```
In [49]: # Network Architecture Diagram
fig, ax = plt.subplots(figsize=(10, 8))
ax.set_xlim(0, 10)
ax.set_ylim(0, 10)
ax.axis('off')

# Define layers
layers = [
    ('Input\n(10 features)', '10 neurons', '#4CAF50'),
    ('Dense + ReLU\n+ Dropout(0.2)', '128 neurons', '#2196F3'),
    ('Dense + ReLU\n+ Dropout(0.2)', '64 neurons', '#2196F3'),
    ('Dense + ReLU', '32 neurons', '#2196F3'),
    ('Dense + Linear\n(Output)', '1 neuron', '#FF9800')
]

n = len(layers)
box_h = 1.2
gap = (10 - n * box_h) / (n + 1)
y_positions = [10 - gap - box_h / 2 - i * (box_h + gap) for i in range(n)]

for idx, (label, size, color) in enumerate(layers):
    y = y_positions[idx]
    rect = plt.Rectangle((2.5, y - box_h/2), 5, box_h,
                        facecolor=color, edgecolor='black',
                        linewidth=2, alpha=0.85, zorder=2)
    ax.add_patch(rect)
    ax.text(5, y + 0.05, label, ha='center', va='center',
           fontsize=11, fontweight='bold', color='white', zorder=3)
    ax.text(8.0, y, size, ha='left', va='center',
           fontsize=10, color='#333333', zorder=3)
    # Draw arrow to next layer
    if idx < n - 1:
        ax.annotate('', xy=(5, y_positions[idx+1] + box_h/2),
                    xytext=(5, y - box_h/2),
                    arrowprops=dict(arrowstyle='->', color='#333333',
```

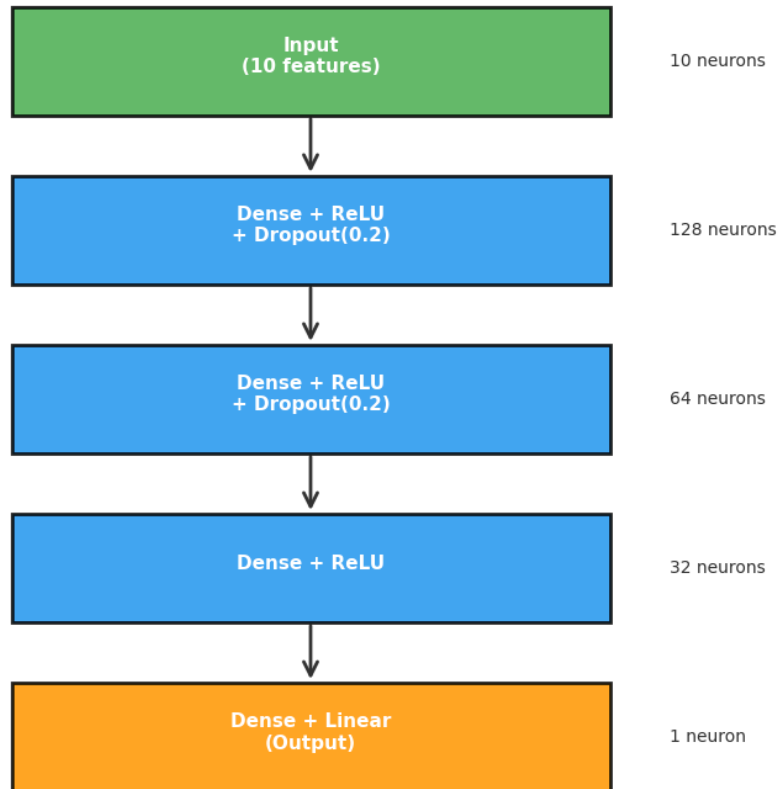
```

        lw=2, mutation_scale=20),
        zorder=1)

ax.set_title('Baseline Model Architecture', fontsize=14, fontweight='bold',
plt.tight_layout()
plt.savefig('plots/network_architecture.png', dpi=150, bbox_inches='tight')
plt.show()

```

Baseline Model Architecture



```

In [50]: early_stop = EarlyStopping(
    monitor="val_loss",
    patience=30,
    restore_best_weights=True
)

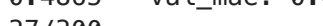
reduce_lr = ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=10,
    min_lr=1e-6,
    verbose=1
)

history = baseline_model.fit(
    X_train_scaled, y_train,
    validation_data=(X_val_scaled, y_val),
    epochs=300,
    batch_size=32,

```

```
callbacks=[early_stop, reduce_lr],  
verbose=1  
)
```

Epoch 1/300
21/21  1s 4ms/step - loss: 32.9375 - mae: 4.5191 - val_loss: 26.0103 - val_mae: 3.8709 - learning_rate: 0.0010
Epoch 2/300
21/21  0s 1ms/step - loss: 18.2955 - mae: 3.1578 - val_loss: 11.4379 - val_mae: 2.5946 - learning_rate: 0.0010
Epoch 3/300
21/21  0s 1ms/step - loss: 8.4654 - mae: 2.1538 - val_loss: 5.2181 - val_mae: 1.6494 - learning_rate: 0.0010
Epoch 4/300
21/21  0s 1ms/step - loss: 4.0017 - mae: 1.3634 - val_loss: 2.7052 - val_mae: 1.0867 - learning_rate: 0.0010
Epoch 5/300
21/21  0s 1ms/step - loss: 2.4793 - mae: 1.0559 - val_loss: 2.1320 - val_mae: 0.9327 - learning_rate: 0.0010
Epoch 6/300
21/21  0s 1ms/step - loss: 2.3928 - mae: 0.9891 - val_loss: 1.6304 - val_mae: 0.8557 - learning_rate: 0.0010
Epoch 7/300
21/21  0s 1ms/step - loss: 2.0059 - mae: 0.9821 - val_loss: 1.4074 - val_mae: 0.7954 - learning_rate: 0.0010
Epoch 8/300
21/21  0s 1ms/step - loss: 1.8993 - mae: 0.8874 - val_loss: 1.3016 - val_mae: 0.7411 - learning_rate: 0.0010
Epoch 9/300
21/21  0s 980us/step - loss: 1.7067 - mae: 0.8781 - val_loss: 1.4109 - val_mae: 0.7157 - learning_rate: 0.0010
Epoch 10/300
21/21  0s 1ms/step - loss: 1.6385 - mae: 0.8905 - val_loss: 1.3194 - val_mae: 0.6918 - learning_rate: 0.0010
Epoch 11/300
21/21  0s 1ms/step - loss: 1.5923 - mae: 0.8139 - val_loss: 1.2460 - val_mae: 0.6909 - learning_rate: 0.0010
Epoch 12/300
21/21  0s 1ms/step - loss: 1.5788 - mae: 0.8480 - val_loss: 1.1898 - val_mae: 0.6605 - learning_rate: 0.0010
Epoch 13/300
21/21  0s 1ms/step - loss: 1.2912 - mae: 0.7732 - val_loss: 0.9986 - val_mae: 0.6213 - learning_rate: 0.0010
Epoch 14/300
21/21  0s 1ms/step - loss: 1.4590 - mae: 0.7875 - val_loss: 0.9674 - val_mae: 0.6077 - learning_rate: 0.0010
Epoch 15/300
21/21  0s 1ms/step - loss: 1.3515 - mae: 0.7965 - val_loss: 1.0430 - val_mae: 0.6175 - learning_rate: 0.0010
Epoch 16/300
21/21  0s 1ms/step - loss: 1.6128 - mae: 0.8453 - val_loss: 0.9940 - val_mae: 0.6036 - learning_rate: 0.0010
Epoch 17/300
21/21  0s 1ms/step - loss: 1.2095 - mae: 0.7268 - val_loss: 0.8097 - val_mae: 0.5726 - learning_rate: 0.0010
Epoch 18/300
21/21  0s 1ms/step - loss: 1.0222 - mae: 0.6884 - val_loss: 1.0435 - val_mae: 0.6306 - learning_rate: 0.0010
Epoch 19/300
21/21  0s 1ms/step - loss: 1.1829 - mae: 0.7416 - val_loss: 1.0435 - val_mae: 0.6306 - learning_rate: 0.0010

ss: 0.9760 - val_mae: 0.6219 - learning_rate: 0.0010
Epoch 20/300
21/21  0s 2ms/step - loss: 1.2706 - mae: 0.7440 - val_loss: 0.6403 - val_mae: 0.5376 - learning_rate: 0.0010
Epoch 21/300
21/21  0s 1ms/step - loss: 1.2725 - mae: 0.7446 - val_loss: 0.8669 - val_mae: 0.6054 - learning_rate: 0.0010
Epoch 22/300
21/21  0s 1ms/step - loss: 1.3013 - mae: 0.7509 - val_loss: 1.0046 - val_mae: 0.6543 - learning_rate: 0.0010
Epoch 23/300
21/21  0s 1ms/step - loss: 1.1282 - mae: 0.7196 - val_loss: 0.9082 - val_mae: 0.6271 - learning_rate: 0.0010
Epoch 24/300
21/21  0s 1ms/step - loss: 1.0377 - mae: 0.6991 - val_loss: 0.7622 - val_mae: 0.5585 - learning_rate: 0.0010
Epoch 25/300
21/21  0s 1ms/step - loss: 0.9719 - mae: 0.6549 - val_loss: 1.0524 - val_mae: 0.6960 - learning_rate: 0.0010
Epoch 26/300
21/21  0s 1ms/step - loss: 1.0247 - mae: 0.6409 - val_loss: 0.7100 - val_mae: 0.5364 - learning_rate: 0.0010
Epoch 27/300
21/21  0s 1ms/step - loss: 1.0952 - mae: 0.6708 - val_loss: 0.7168 - val_mae: 0.5653 - learning_rate: 0.0010
Epoch 28/300
21/21  0s 1ms/step - loss: 0.7941 - mae: 0.6098 - val_loss: 0.8048 - val_mae: 0.5991 - learning_rate: 0.0010
Epoch 29/300
21/21  0s 1ms/step - loss: 1.0586 - mae: 0.6577 - val_loss: 1.1023 - val_mae: 0.7130 - learning_rate: 0.0010
Epoch 30/300
21/21  0s 1ms/step - loss: 1.1032 - mae: 0.6682 - val_loss: 0.6326 - val_mae: 0.5269 - learning_rate: 0.0010
Epoch 31/300
21/21  0s 1ms/step - loss: 0.8487 - mae: 0.6003 - val_loss: 0.5270 - val_mae: 0.4855 - learning_rate: 0.0010
Epoch 32/300
21/21  0s 1ms/step - loss: 0.8574 - mae: 0.6061 - val_loss: 0.4940 - val_mae: 0.4743 - learning_rate: 0.0010
Epoch 33/300
21/21  0s 1ms/step - loss: 0.8564 - mae: 0.6047 - val_loss: 0.5547 - val_mae: 0.4935 - learning_rate: 0.0010
Epoch 34/300
21/21  0s 1ms/step - loss: 0.8631 - mae: 0.6020 - val_loss: 0.4017 - val_mae: 0.4367 - learning_rate: 0.0010
Epoch 35/300
21/21  0s 1ms/step - loss: 0.7999 - mae: 0.6143 - val_loss: 0.5692 - val_mae: 0.5137 - learning_rate: 0.0010
Epoch 36/300
21/21  0s 994us/step - loss: 0.7862 - mae: 0.5996 - val_loss: 0.4865 - val_mae: 0.4713 - learning_rate: 0.0010
Epoch 37/300
21/21  0s 1000us/step - loss: 0.7505 - mae: 0.5692 - val_loss: 0.4270 - val_mae: 0.4316 - learning_rate: 0.0010
Epoch 38/300

21/21  0s 1ms/step - loss: 0.6877 - mae: 0.5702 - val_loss: 0.4265 - val_mae: 0.4398 - learning_rate: 0.0010
Epoch 39/300

21/21  0s 1ms/step - loss: 0.5483 - mae: 0.5078 - val_loss: 0.3606 - val_mae: 0.4110 - learning_rate: 0.0010
Epoch 40/300

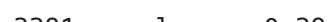
21/21  0s 2ms/step - loss: 0.8436 - mae: 0.5702 - val_loss: 0.3852 - val_mae: 0.4095 - learning_rate: 0.0010
Epoch 41/300

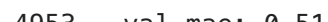
21/21  0s 1ms/step - loss: 0.7530 - mae: 0.5439 - val_loss: 0.4483 - val_mae: 0.4728 - learning_rate: 0.0010
Epoch 42/300

21/21  0s 1ms/step - loss: 0.6017 - mae: 0.5249 - val_loss: 0.5704 - val_mae: 0.5121 - learning_rate: 0.0010
Epoch 43/300

21/21  0s 1000us/step - loss: 0.6488 - mae: 0.5194 - val_loss: 0.4232 - val_mae: 0.4354 - learning_rate: 0.0010
Epoch 44/300

21/21  0s 1ms/step - loss: 0.5950 - mae: 0.5066 - val_loss: 0.4518 - val_mae: 0.4258 - learning_rate: 0.0010
Epoch 45/300

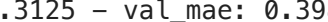
21/21  0s 1ms/step - loss: 0.6263 - mae: 0.5446 - val_loss: 0.3391 - val_mae: 0.3982 - learning_rate: 0.0010
Epoch 46/300

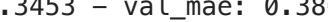
21/21  0s 1ms/step - loss: 0.7016 - mae: 0.5248 - val_loss: 0.4953 - val_mae: 0.5128 - learning_rate: 0.0010
Epoch 47/300

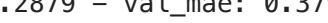
21/21  0s 1ms/step - loss: 0.6649 - mae: 0.5232 - val_loss: 0.3324 - val_mae: 0.3879 - learning_rate: 0.0010
Epoch 48/300

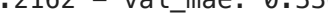
21/21  0s 1ms/step - loss: 0.7477 - mae: 0.5305 - val_loss: 0.2603 - val_mae: 0.3583 - learning_rate: 0.0010
Epoch 49/300

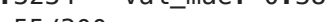
21/21  0s 1000us/step - loss: 0.6514 - mae: 0.5401 - val_loss: 0.4190 - val_mae: 0.4380 - learning_rate: 0.0010
Epoch 50/300

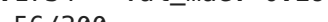
21/21  0s 1ms/step - loss: 0.6563 - mae: 0.5285 - val_loss: 0.3125 - val_mae: 0.3984 - learning_rate: 0.0010
Epoch 51/300

21/21  0s 1ms/step - loss: 0.5104 - mae: 0.4938 - val_loss: 0.3453 - val_mae: 0.3861 - learning_rate: 0.0010
Epoch 52/300

21/21  0s 1ms/step - loss: 0.5200 - mae: 0.4759 - val_loss: 0.2879 - val_mae: 0.3796 - learning_rate: 0.0010
Epoch 53/300

21/21  0s 1ms/step - loss: 0.5657 - mae: 0.4966 - val_loss: 0.2162 - val_mae: 0.3307 - learning_rate: 0.0010
Epoch 54/300

21/21  0s 1ms/step - loss: 0.5566 - mae: 0.5018 - val_loss: 0.3234 - val_mae: 0.3849 - learning_rate: 0.0010
Epoch 55/300

21/21  0s 1ms/step - loss: 0.5808 - mae: 0.5071 - val_loss: 0.1734 - val_mae: 0.2913 - learning_rate: 0.0010
Epoch 56/300

21/21  0s 1ms/step - loss: 0.4829 - mae: 0.4667 - val_loss: 0.2463 - val_mae: 0.3642 - learning_rate: 0.0010

Epoch 57/300
21/21  0s 992us/step - loss: 0.5658 - mae: 0.4827 - val_loss: 0.2298 - val_mae: 0.3267 - learning_rate: 0.0010
Epoch 58/300
21/21  0s 1ms/step - loss: 0.6742 - mae: 0.5124 - val_loss: 0.2815 - val_mae: 0.3815 - learning_rate: 0.0010
Epoch 59/300
21/21  0s 1ms/step - loss: 0.4680 - mae: 0.4670 - val_loss: 0.2925 - val_mae: 0.3838 - learning_rate: 0.0010
Epoch 60/300
21/21  0s 979us/step - loss: 0.5400 - mae: 0.4769 - val_loss: 0.2996 - val_mae: 0.3641 - learning_rate: 0.0010
Epoch 61/300
21/21  0s 984us/step - loss: 0.4551 - mae: 0.4619 - val_loss: 0.2809 - val_mae: 0.3557 - learning_rate: 0.0010
Epoch 62/300
21/21  0s 985us/step - loss: 0.5395 - mae: 0.4869 - val_loss: 0.2678 - val_mae: 0.3317 - learning_rate: 0.0010
Epoch 63/300
21/21  0s 973us/step - loss: 0.4812 - mae: 0.4535 - val_loss: 0.2683 - val_mae: 0.3813 - learning_rate: 0.0010
Epoch 64/300
21/21  0s 962us/step - loss: 0.5387 - mae: 0.4867 - val_loss: 0.5413 - val_mae: 0.5121 - learning_rate: 0.0010
Epoch 65/300
1/21  0s 5ms/step - loss: 0.2783 - mae: 0.3799
Epoch 65: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
21/21  0s 1ms/step - loss: 0.3695 - mae: 0.4369 - val_loss: 0.4882 - val_mae: 0.4620 - learning_rate: 0.0010
Epoch 66/300
21/21  0s 995us/step - loss: 0.5558 - mae: 0.4708 - val_loss: 0.2112 - val_mae: 0.3232 - learning_rate: 5.0000e-04
Epoch 67/300
21/21  0s 1ms/step - loss: 0.4519 - mae: 0.4667 - val_loss: 0.2515 - val_mae: 0.3478 - learning_rate: 5.0000e-04
Epoch 68/300
21/21  0s 1ms/step - loss: 0.4279 - mae: 0.4521 - val_loss: 0.3081 - val_mae: 0.3827 - learning_rate: 5.0000e-04
Epoch 69/300
21/21  0s 986us/step - loss: 0.6501 - mae: 0.4892 - val_loss: 0.2291 - val_mae: 0.3347 - learning_rate: 5.0000e-04
Epoch 70/300
21/21  0s 2ms/step - loss: 0.4726 - mae: 0.4479 - val_loss: 0.2324 - val_mae: 0.3254 - learning_rate: 5.0000e-04
Epoch 71/300
21/21  0s 1ms/step - loss: 0.4893 - mae: 0.4646 - val_loss: 0.2528 - val_mae: 0.3463 - learning_rate: 5.0000e-04
Epoch 72/300
21/21  0s 987us/step - loss: 0.5060 - mae: 0.4536 - val_loss: 0.2534 - val_mae: 0.3643 - learning_rate: 5.0000e-04
Epoch 73/300
21/21  0s 998us/step - loss: 0.4438 - mae: 0.4219 - val_loss: 0.1519 - val_mae: 0.2867 - learning_rate: 5.0000e-04
Epoch 74/300
21/21  0s 1ms/step - loss: 0.3986 - mae: 0.4348 - val_loss: 0.1827 - val_mae: 0.3114 - learning_rate: 5.0000e-04

Epoch 75/300
21/21  0s 993us/step - loss: 0.4435 - mae: 0.4447 - val_loss: 0.1634 - val_mae: 0.3030 - learning_rate: 5.0000e-04
Epoch 76/300
21/21  0s 973us/step - loss: 0.4612 - mae: 0.4441 - val_loss: 0.2747 - val_mae: 0.3829 - learning_rate: 5.0000e-04
Epoch 77/300
21/21  0s 1ms/step - loss: 0.4282 - mae: 0.4463 - val_loss: 0.1407 - val_mae: 0.2761 - learning_rate: 5.0000e-04
Epoch 78/300
21/21  0s 1ms/step - loss: 0.4047 - mae: 0.4461 - val_loss: 0.1604 - val_mae: 0.2930 - learning_rate: 5.0000e-04
Epoch 79/300
21/21  0s 990us/step - loss: 0.3646 - mae: 0.4029 - val_loss: 0.1730 - val_mae: 0.3081 - learning_rate: 5.0000e-04
Epoch 80/300
21/21  0s 994us/step - loss: 0.4039 - mae: 0.4310 - val_loss: 0.2088 - val_mae: 0.3332 - learning_rate: 5.0000e-04
Epoch 81/300
21/21  0s 1ms/step - loss: 0.5887 - mae: 0.4742 - val_loss: 0.2384 - val_mae: 0.3644 - learning_rate: 5.0000e-04
Epoch 82/300
21/21  0s 990us/step - loss: 0.3710 - mae: 0.4176 - val_loss: 0.2321 - val_mae: 0.3385 - learning_rate: 5.0000e-04
Epoch 83/300
21/21  0s 970us/step - loss: 0.4231 - mae: 0.4426 - val_loss: 0.1656 - val_mae: 0.3006 - learning_rate: 5.0000e-04
Epoch 84/300
21/21  0s 1ms/step - loss: 0.3582 - mae: 0.3872 - val_loss: 0.1527 - val_mae: 0.3034 - learning_rate: 5.0000e-04
Epoch 85/300
21/21  0s 982us/step - loss: 0.4160 - mae: 0.4277 - val_loss: 0.1481 - val_mae: 0.2902 - learning_rate: 5.0000e-04
Epoch 86/300
21/21  0s 977us/step - loss: 0.3488 - mae: 0.4104 - val_loss: 0.1435 - val_mae: 0.2906 - learning_rate: 5.0000e-04
Epoch 87/300
1/21  0s 5ms/step - loss: 0.5585 - mae: 0.4925
Epoch 87: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
21/21  0s 997us/step - loss: 0.4149 - mae: 0.4241 - val_loss: 0.3048 - val_mae: 0.4055 - learning_rate: 5.0000e-04
Epoch 88/300
21/21  0s 980us/step - loss: 0.3912 - mae: 0.4229 - val_loss: 0.1422 - val_mae: 0.2802 - learning_rate: 2.5000e-04
Epoch 89/300
21/21  0s 990us/step - loss: 0.4946 - mae: 0.4485 - val_loss: 0.1800 - val_mae: 0.3213 - learning_rate: 2.5000e-04
Epoch 90/300
21/21  0s 977us/step - loss: 0.4153 - mae: 0.4255 - val_loss: 0.1495 - val_mae: 0.2949 - learning_rate: 2.5000e-04
Epoch 91/300
21/21  0s 988us/step - loss: 0.4991 - mae: 0.4422 - val_loss: 0.2242 - val_mae: 0.3494 - learning_rate: 2.5000e-04
Epoch 92/300
21/21  0s 974us/step - loss: 0.4209 - mae: 0.4352 - val_loss: 0.1208 - val_mae: 0.2605 - learning_rate: 2.5000e-04

```

Epoch 93/300
21/21  0s 1000us/step - loss: 0.3490 - mae: 0.3956 - val_
_loss: 0.1495 - val_mae: 0.2966 - learning_rate: 2.5000e-04
Epoch 94/300
21/21  0s 986us/step - loss: 0.3686 - mae: 0.4068 - val_
_loss: 0.1454 - val_mae: 0.2917 - learning_rate: 2.5000e-04
Epoch 95/300
21/21  0s 975us/step - loss: 0.4126 - mae: 0.4377 - val_
_loss: 0.1774 - val_mae: 0.3135 - learning_rate: 2.5000e-04
Epoch 96/300
21/21  0s 987us/step - loss: 0.3282 - mae: 0.3792 - val_
_loss: 0.1238 - val_mae: 0.2631 - learning_rate: 2.5000e-04
Epoch 97/300
21/21  0s 975us/step - loss: 0.4213 - mae: 0.4359 - val_
_loss: 0.1673 - val_mae: 0.3089 - learning_rate: 2.5000e-04
Epoch 98/300
21/21  0s 2ms/step - loss: 0.4710 - mae: 0.4569 - val_lo
ss: 0.1679 - val_mae: 0.3069 - learning_rate: 2.5000e-04
Epoch 99/300
21/21  0s 1000us/step - loss: 0.3509 - mae: 0.3967 - val_
_loss: 0.2115 - val_mae: 0.3399 - learning_rate: 2.5000e-04
Epoch 100/300
21/21  0s 988us/step - loss: 0.3881 - mae: 0.3993 - val_
_loss: 0.1333 - val_mae: 0.2744 - learning_rate: 2.5000e-04
Epoch 101/300
21/21  0s 981us/step - loss: 0.3793 - mae: 0.4430 - val_
_loss: 0.1692 - val_mae: 0.3141 - learning_rate: 2.5000e-04
Epoch 102/300
1/21  0s 5ms/step - loss: 0.2131 - mae: 0.3652
Epoch 102: ReduceLRonPlateau reducing learning rate to 0.000125000005937181
4.
21/21  0s 998us/step - loss: 0.3730 - mae: 0.4003 - val_
_loss: 0.1667 - val_mae: 0.3041 - learning_rate: 2.5000e-04
Epoch 103/300
21/21  0s 990us/step - loss: 0.4011 - mae: 0.4108 - val_
_loss: 0.1253 - val_mae: 0.2667 - learning_rate: 1.2500e-04
Epoch 104/300
21/21  0s 979us/step - loss: 0.4017 - mae: 0.4253 - val_
_loss: 0.2058 - val_mae: 0.3298 - learning_rate: 1.2500e-04
Epoch 105/300
21/21  0s 1ms/step - loss: 0.3396 - mae: 0.3939 - val_lo
ss: 0.1435 - val_mae: 0.2823 - learning_rate: 1.2500e-04
Epoch 106/300
21/21  0s 992us/step - loss: 0.3675 - mae: 0.4251 - val_
_loss: 0.1439 - val_mae: 0.2840 - learning_rate: 1.2500e-04
Epoch 107/300
21/21  0s 972us/step - loss: 0.4403 - mae: 0.4267 - val_
_loss: 0.1377 - val_mae: 0.2800 - learning_rate: 1.2500e-04
Epoch 108/300
21/21  0s 971us/step - loss: 0.3892 - mae: 0.4105 - val_
_loss: 0.1345 - val_mae: 0.2764 - learning_rate: 1.2500e-04
Epoch 109/300
21/21  0s 989us/step - loss: 0.3878 - mae: 0.4259 - val_
_loss: 0.1340 - val_mae: 0.2768 - learning_rate: 1.2500e-04
Epoch 110/300
21/21  0s 982us/step - loss: 0.3928 - mae: 0.4063 - val_

```

```

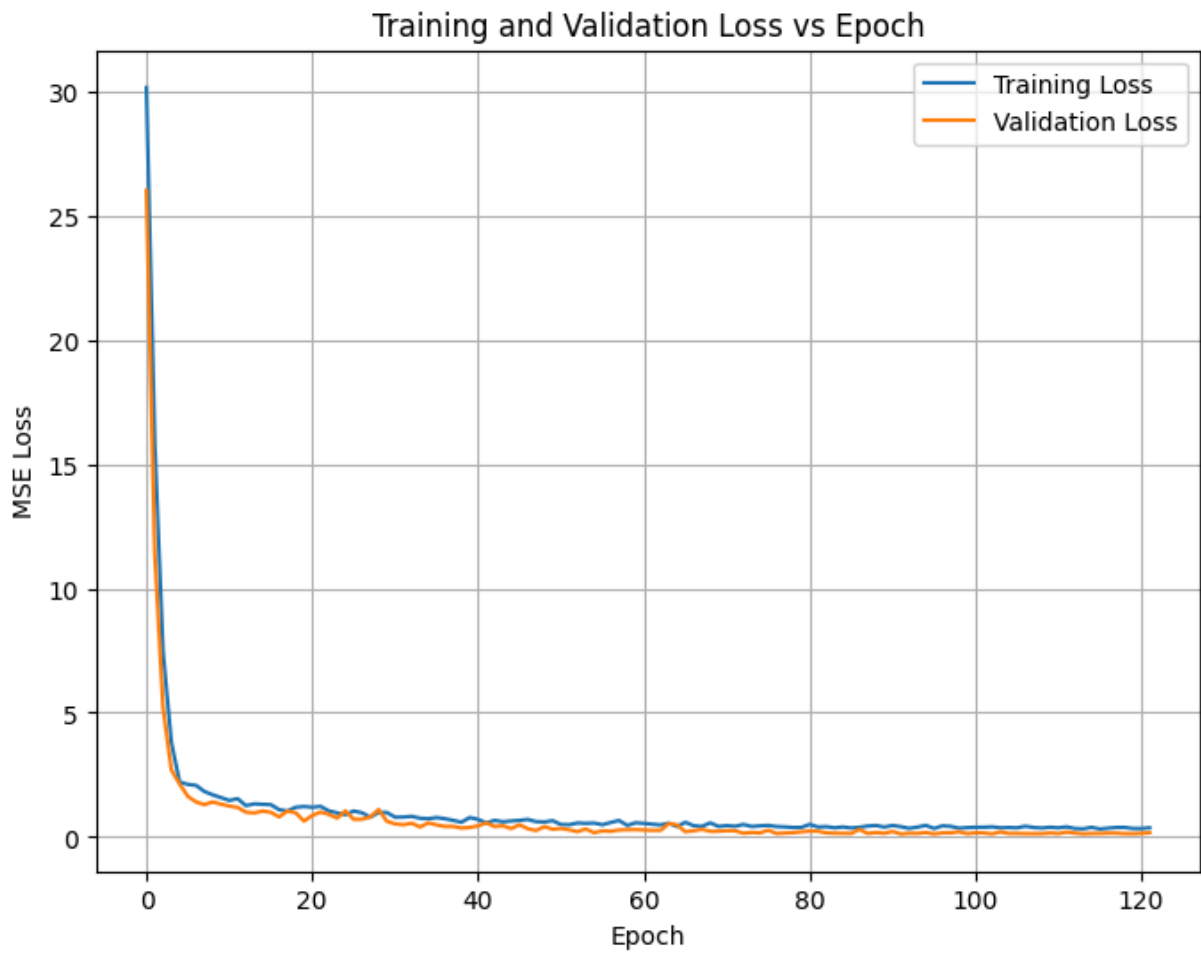
loss: 0.1621 - val_mae: 0.3019 - learning_rate: 1.2500e-04
Epoch 111/300
21/21 ----- 0s 994us/step - loss: 0.3560 - mae: 0.3942 - val_
loss: 0.1370 - val_mae: 0.2744 - learning_rate: 1.2500e-04
Epoch 112/300
1/21 ----- 0s 5ms/step - loss: 0.2631 - mae: 0.3704
Epoch 112: ReduceLRonPlateau reducing learning rate to 6.25000029685907e-05.
21/21 ----- 0s 997us/step - loss: 0.4172 - mae: 0.4192 - val_
loss: 0.2006 - val_mae: 0.3261 - learning_rate: 1.2500e-04
Epoch 113/300
21/21 ----- 0s 985us/step - loss: 0.3529 - mae: 0.4070 - val_
loss: 0.1549 - val_mae: 0.2890 - learning_rate: 6.2500e-05
Epoch 114/300
21/21 ----- 0s 984us/step - loss: 0.3026 - mae: 0.3755 - val_
loss: 0.1262 - val_mae: 0.2640 - learning_rate: 6.2500e-05
Epoch 115/300
21/21 ----- 0s 976us/step - loss: 0.4146 - mae: 0.4254 - val_
loss: 0.1408 - val_mae: 0.2802 - learning_rate: 6.2500e-05
Epoch 116/300
21/21 ----- 0s 974us/step - loss: 0.2979 - mae: 0.3765 - val_
loss: 0.1408 - val_mae: 0.2818 - learning_rate: 6.2500e-05
Epoch 117/300
21/21 ----- 0s 977us/step - loss: 0.3987 - mae: 0.4175 - val_
loss: 0.1568 - val_mae: 0.2954 - learning_rate: 6.2500e-05
Epoch 118/300
21/21 ----- 0s 959us/step - loss: 0.4257 - mae: 0.4275 - val_
loss: 0.1533 - val_mae: 0.2934 - learning_rate: 6.2500e-05
Epoch 119/300
21/21 ----- 0s 984us/step - loss: 0.4070 - mae: 0.4103 - val_
loss: 0.1359 - val_mae: 0.2777 - learning_rate: 6.2500e-05
Epoch 120/300
21/21 ----- 0s 1ms/step - loss: 0.3334 - mae: 0.3923 - val_lo
ss: 0.1314 - val_mae: 0.2740 - learning_rate: 6.2500e-05
Epoch 121/300
21/21 ----- 0s 988us/step - loss: 0.3579 - mae: 0.3851 - val_
loss: 0.1466 - val_mae: 0.2879 - learning_rate: 6.2500e-05
Epoch 122/300
1/21 ----- 0s 5ms/step - loss: 0.3935 - mae: 0.4617
Epoch 122: ReduceLRonPlateau reducing learning rate to 3.125000148429535e-05.
21/21 ----- 0s 2ms/step - loss: 0.3693 - mae: 0.4205 - val_lo
ss: 0.1733 - val_mae: 0.3087 - learning_rate: 6.2500e-05

```

```

In [51]: # Train & Val Loss v/s Epoch Plot
plt.figure(figsize=(8, 6))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("Training and Validation Loss vs Epoch")
plt.legend()
plt.grid(True)
plt.savefig('plots/training_and_validation_loss_vs_epoch.png', dpi=150, bbox
plt.show()

```



Overfitting Check

The plot above shows training and validation loss as a function of epoch. The key indicator that the network is **not overfitting** is that the validation loss tracks closely with the training loss and does not diverge upwards while the training loss continues decreasing.

Additionally, two callbacks are used to prevent overfitting:

- **EarlyStopping** (`patience=30`): Stops training if validation loss does not improve for 30 consecutive epochs, and restores the weights from the best epoch.
- **ReduceLROnPlateau** (`patience=10` , `factor=0.5`): Reduces the learning rate by half if the validation loss plateaus for 10 epochs, allowing finer convergence without overshooting.

Together, these mechanisms ensure the model does not memorise the training data and instead learns generalisable patterns.

```
In [52]: y_pred = baseline_model.predict(X_test_scaled).flatten()

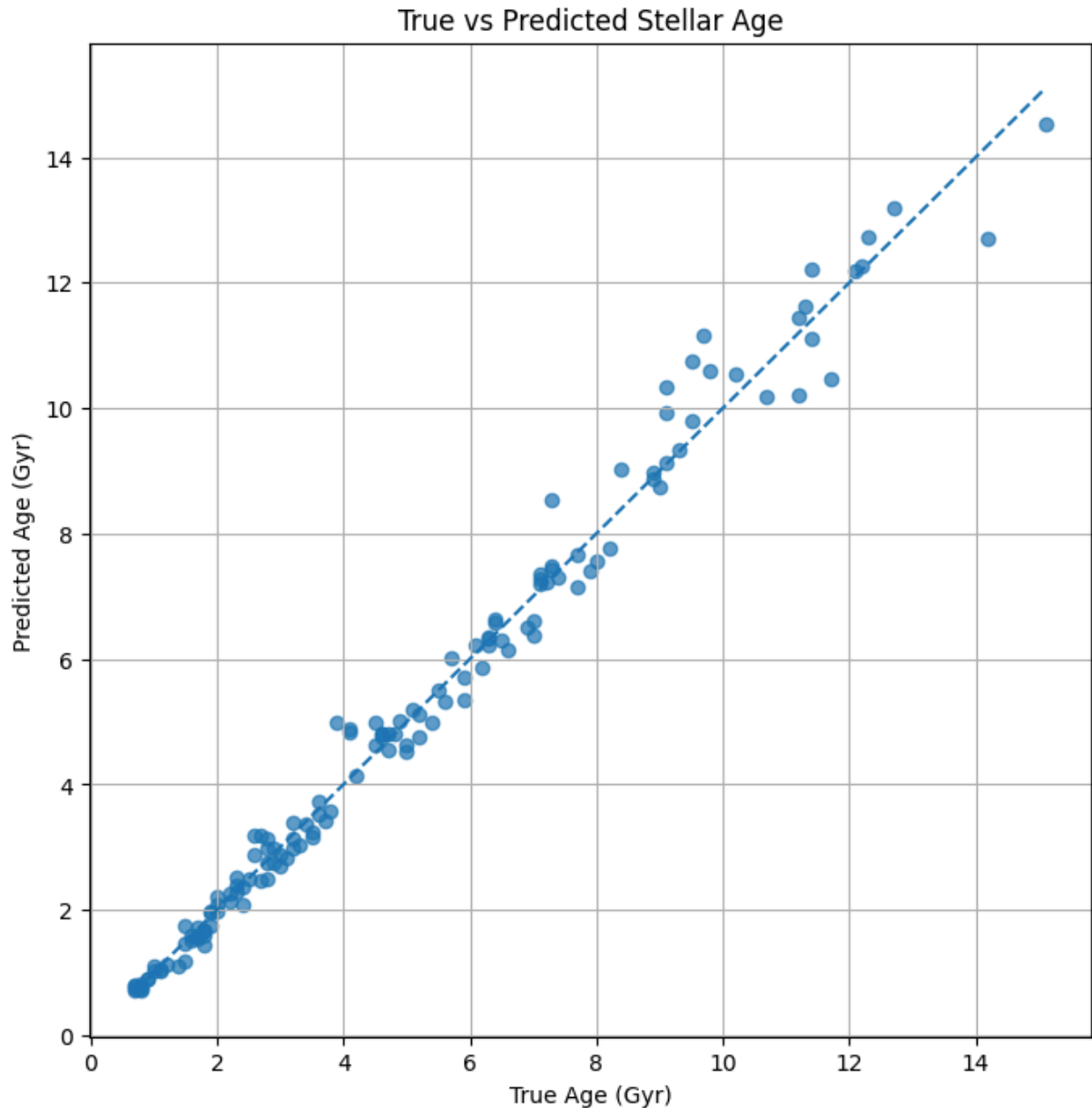
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
```

```
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Test MSE : {mse:.4f}")
print(f"Test RMSE : {rmse:.4f}")
print(f"Test MAE : {mae:.4f}")
print(f"Test R^2 : {r2:.4f}")
```

5/5  0s 5ms/step

```
Test MSE : 0.1714
Test RMSE : 0.4140
Test MAE : 0.2785
Test R^2 : 0.9852
```

```
In [53]: # True v/s Predicted Stellar Age
plt.figure(figsize=(8, 8))
plt.scatter(y_test, y_pred, alpha=0.7)
min_val = min(y_test.min(), y_pred.min())
max_val = max(y_test.max(), y_pred.max())
plt.plot([min_val, max_val], [min_val, max_val], linestyle="--")
plt.xlabel("True Age (Gyr)")
plt.ylabel("Predicted Age (Gyr)")
plt.title("True vs Predicted Stellar Age")
plt.grid(True)
plt.savefig('plots/true_vs_predicted_stellar_age.png', dpi=150, bbox_inches=
plt.show()
```



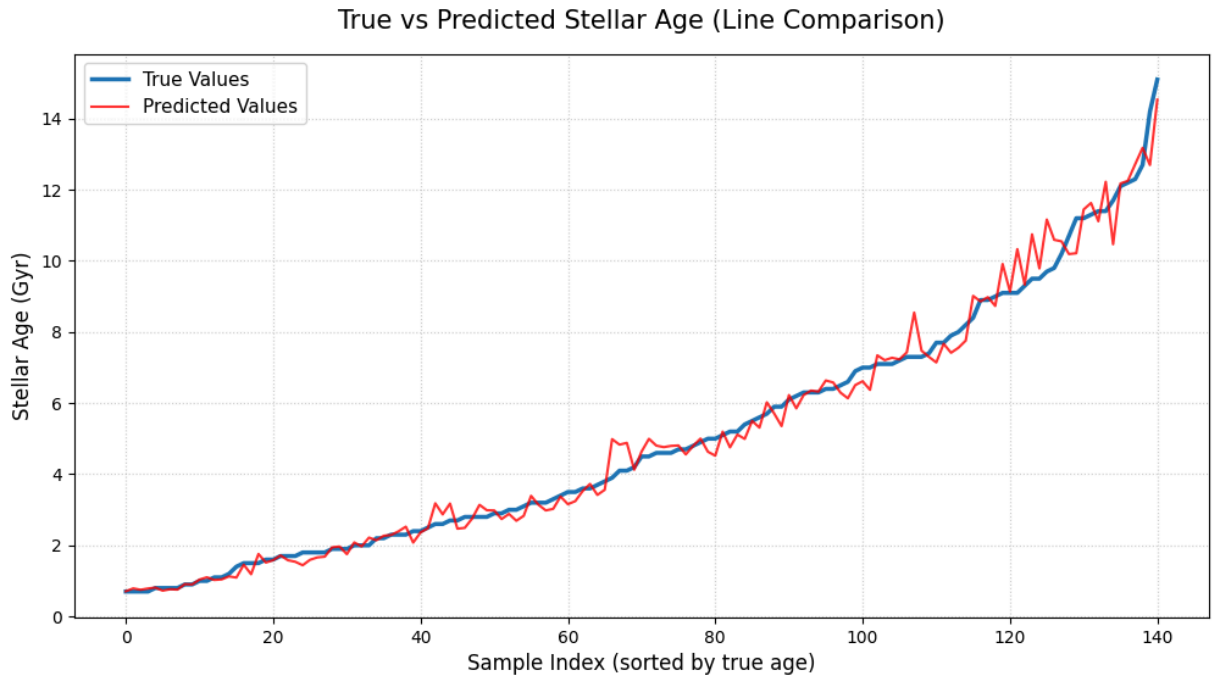
```
In [54]: # True vs Predicted Stellar Age (Line Comparison)
y_test_flat = y_test.values.flatten()
y_pred_flat = y_pred.flatten()

# Sort for cleaner visualization
sorted_idx = np.argsort(y_test_flat)
y_test_sorted = y_test_flat[sorted_idx]
y_pred_sorted = y_pred_flat[sorted_idx]

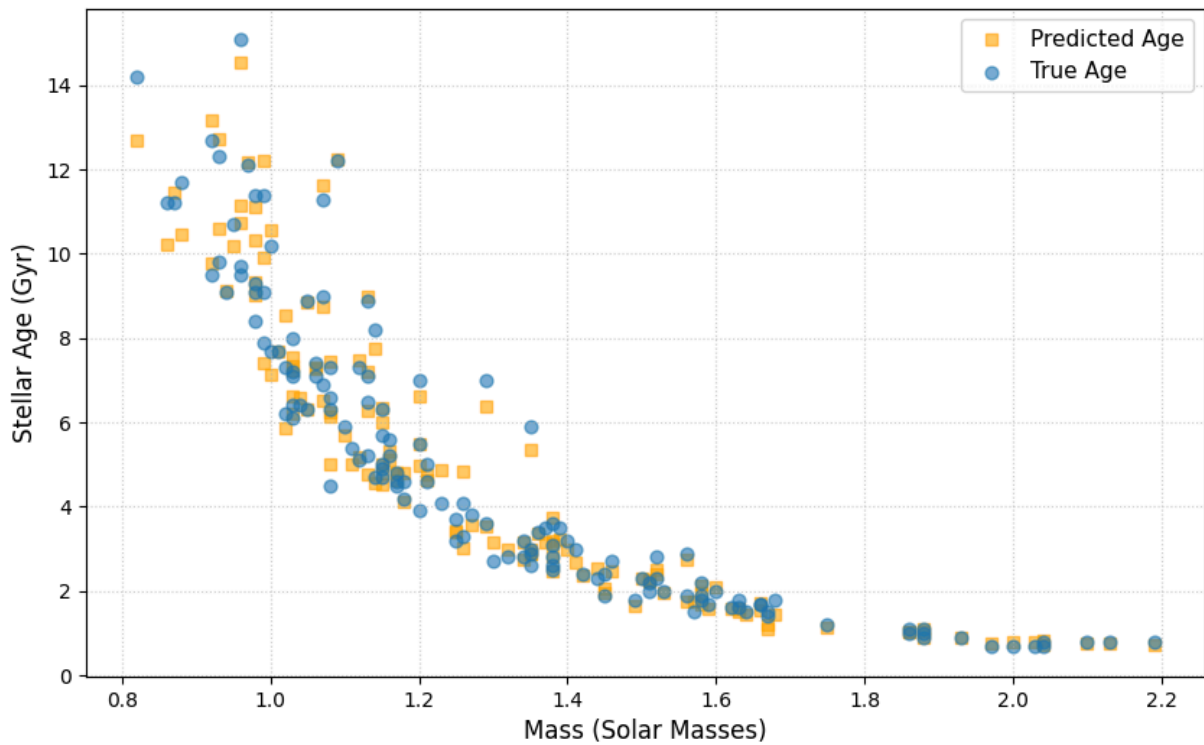
plt.figure(figsize=(12, 6))
plt.plot(y_test_sorted, label="True Values", color='#1f77b4', linewidth=2.5)
plt.plot(y_pred_sorted, label="Predicted Values", color='red', linewidth=1.5)
plt.xlabel("Sample Index (sorted by true age)", fontsize=12)
plt.ylabel("Stellar Age (Gyr)", fontsize=12)
plt.title("True vs Predicted Stellar Age (Line Comparison)", fontsize=15, pad=10)
plt.legend(fontsize=11)
plt.grid(True, linestyle=':', alpha=0.6)
plt.savefig("plots/true_vs_predicted_line_comparison.png", dpi=150, bbox_inches='tight')
```

```
plt.show()

# True and Predicted Stellar Age vs Mass
plt.figure(figsize=(10, 6))
plt.scatter(X_test["mass"], y_pred, alpha=0.6, label='Predicted Age', color=
plt.scatter(X_test["mass"], y_test, alpha=0.6, label='True Age', color='#1f7
plt.xlabel("Mass (Solar Masses)", fontsize=12)
plt.ylabel("Stellar Age (Gyr)", fontsize=12)
plt.title("True and Predicted Stellar Age vs Mass", fontsize=15, pad=15)
plt.legend(fontsize=11)
plt.grid(True, linestyle=':', alpha=0.6)
plt.savefig('plots/stellar_age_vs_mass.png', dpi=150, bbox_inches='tight')
plt.show()
```

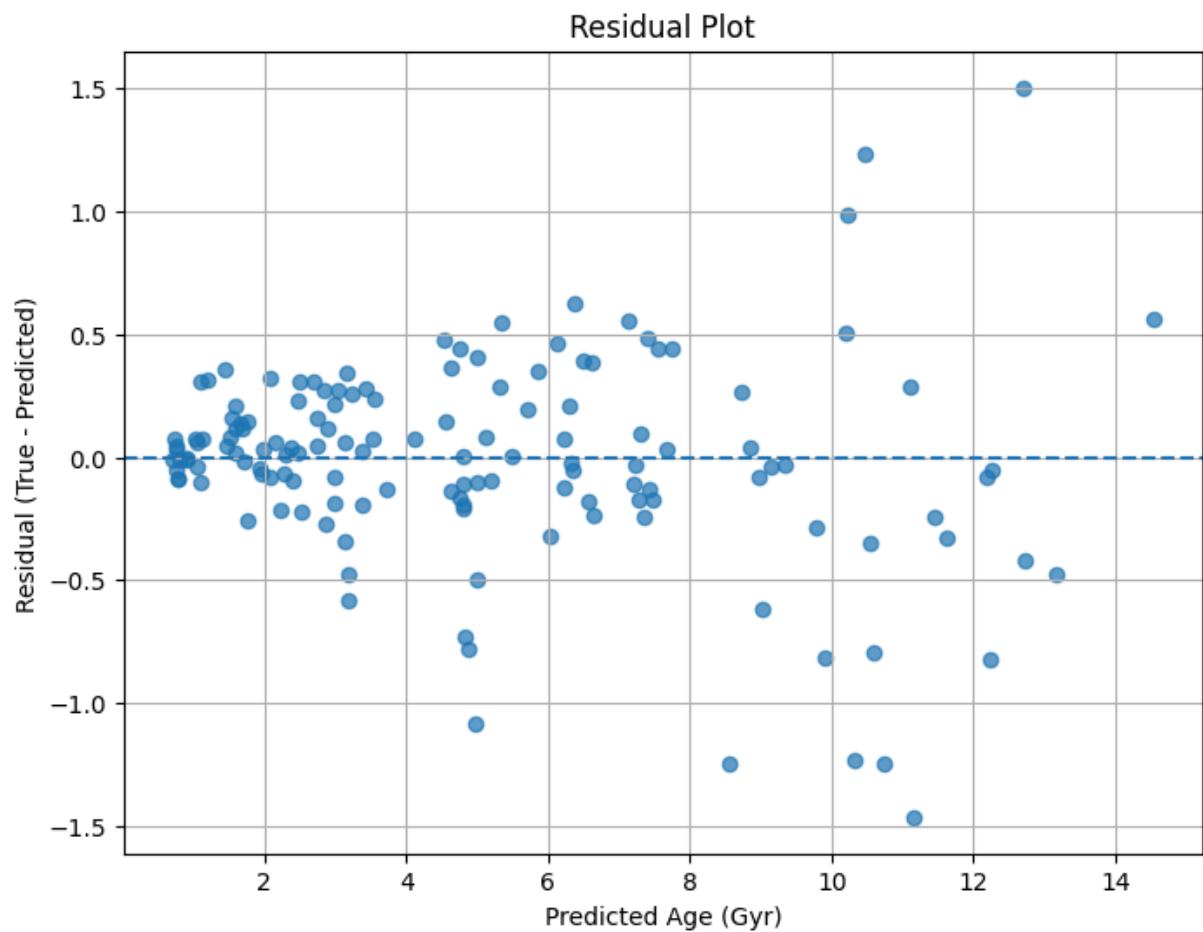


True and Predicted Stellar Age vs Mass

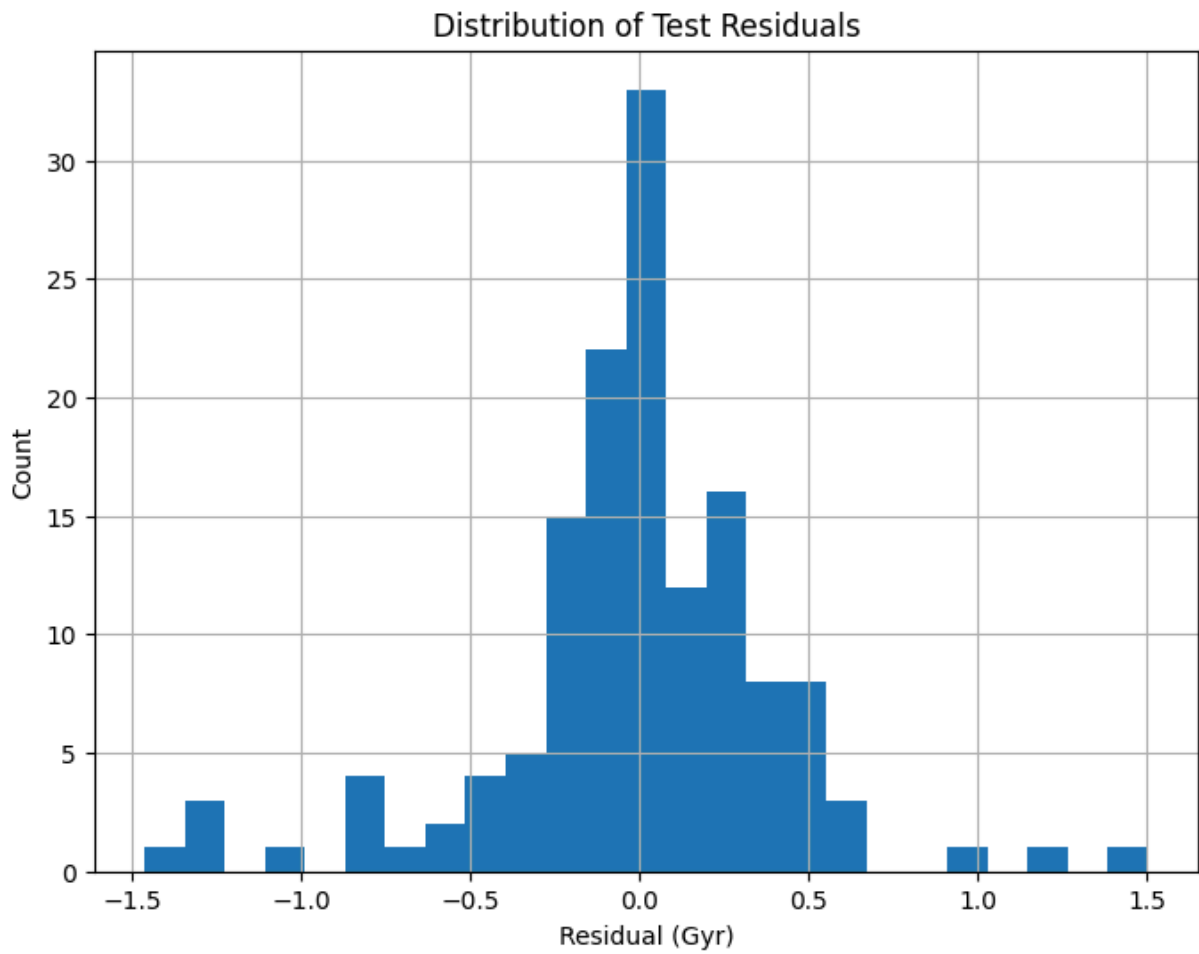


```
In [55]: residuals = y_test - y_pred

plt.figure(figsize=(8, 6))
plt.scatter(y_pred, residuals, alpha=0.7)
plt.axhline(0, linestyle="--")
plt.xlabel("Predicted Age (Gyr)")
plt.ylabel("Residual (True - Predicted)")
plt.title("Residual Plot")
plt.grid(True)
plt.savefig('plots/residual_plot.png', dpi=150, bbox_inches='tight')
plt.show()
```

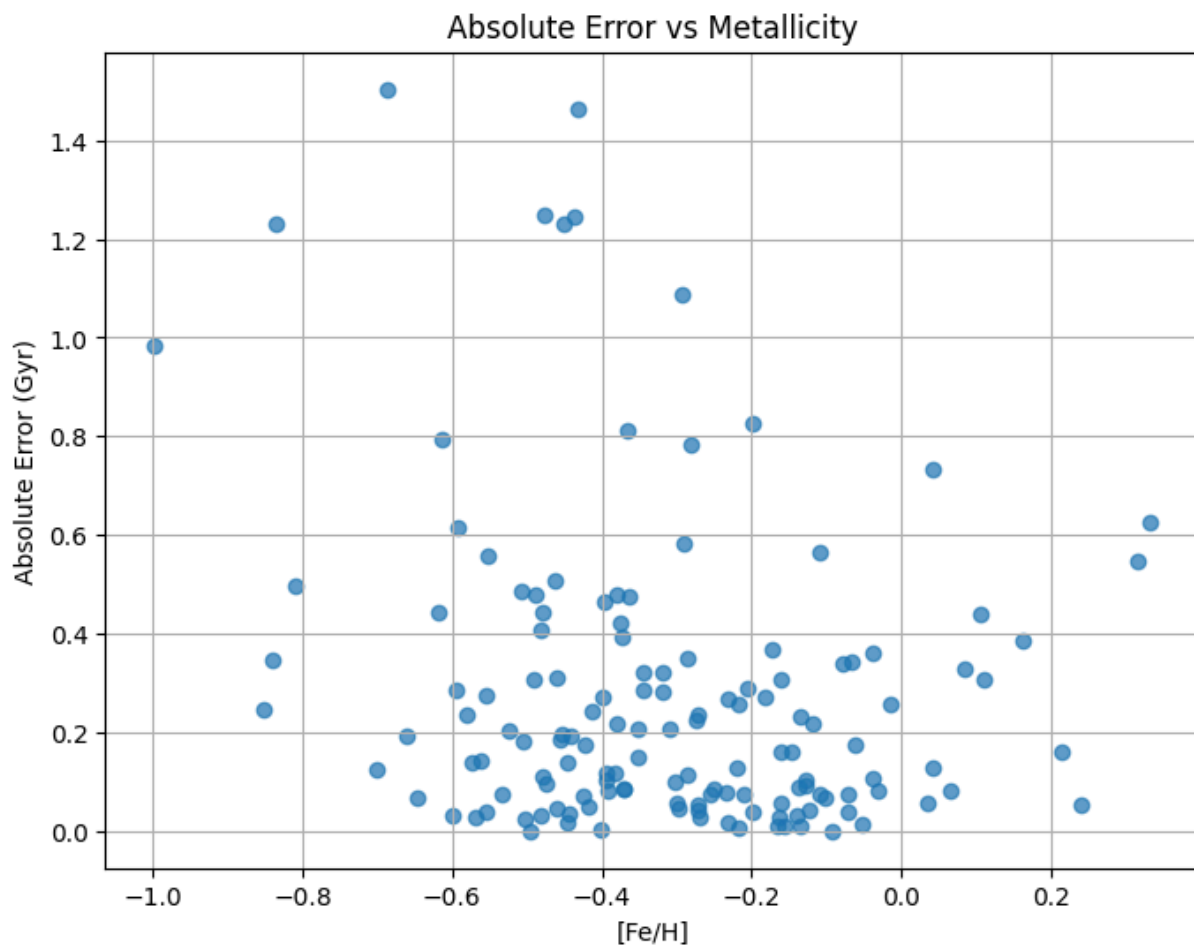


```
In [56]: plt.figure(figsize=(8, 6))
plt.hist(residuals, bins=25)
plt.xlabel("Residual (Gyr)")
plt.ylabel("Count")
plt.title("Distribution of Test Residuals")
plt.grid(True)
plt.savefig('plots/distribution_of_test_residuals.png', dpi=150, bbox_inches=
plt.show()
```



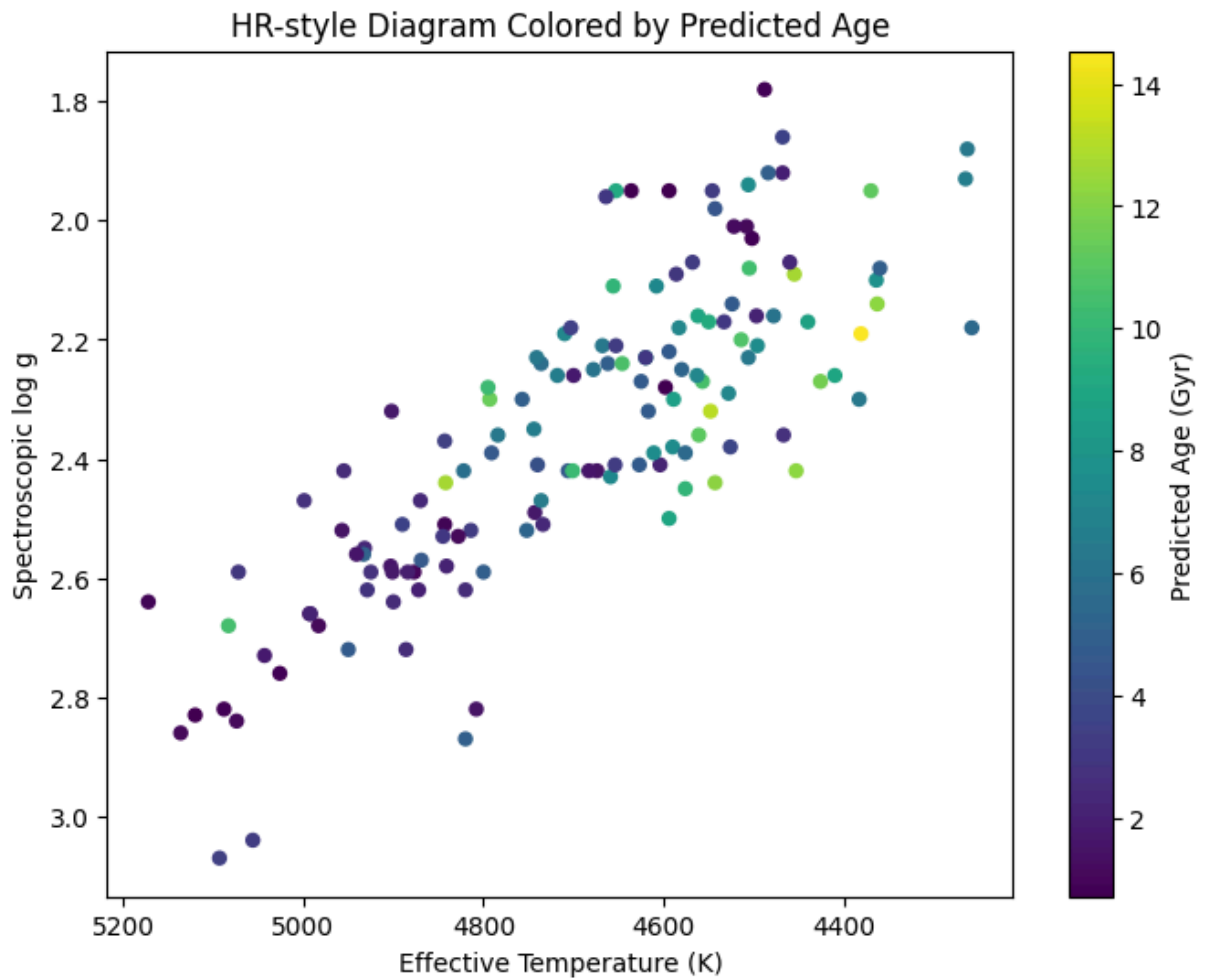
```
In [57]: test_results = X_test.copy()
test_results["true_age"] = y_test.values
test_results["pred_age"] = y_pred
test_results["abs_error"] = np.abs(test_results["true_age"] - test_results["pred_age"])

plt.figure(figsize=(8, 6))
plt.scatter(test_results["feh"], test_results["abs_error"], alpha=0.7)
plt.xlabel("[Fe/H]")
plt.ylabel("Absolute Error (Gyr)")
plt.title("Absolute Error vs Metallicity")
plt.grid(True)
plt.savefig('plots/absolute_error_vs_metallicity.png', dpi=150, bbox_inches='tight')
plt.show()
```



```
In [58]: test_results["teff"] = X_test["teff"].values
test_results["logg_spec"] = X_test["logg_spec"].values

plt.figure(figsize=(8, 6))
plt.scatter(test_results["teff"], test_results["logg_spec"], c=test_results["age"])
plt.gca().invert_xaxis()
plt.gca().invert_yaxis()
plt.xlabel("Effective Temperature (K)")
plt.ylabel("Spectroscopic log g")
plt.title("HR-style Diagram Colored by Predicted Age")
plt.colorbar(label="Predicted Age (Gyr)")
plt.savefig('plots/hr_style_diagram_colored_by_predicted_age.png', dpi=150,
plt.show()
```



```
In [59]: def build_model(hidden_units=[64, 32], dropout_rate=0.2, learning_rate=1e-3)
          model = Sequential()
          model.add(Input(shape=(X_train_scaled.shape[1],)))

          for units in hidden_units:
              model.add(Dense(units, activation="relu"))
              model.add(Dropout(dropout_rate))

          model.add(Dense(1, activation="linear"))

          model.compile(
              optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),
              loss="mse",
              metrics=["mae"]
          )
          return model

          architectures = {
              "small": [32, 16],
              "medium": [128, 64, 32],
              "large": [256, 128, 64, 32]
          }

          comparison_results = []
```

```

for name, arch in architectures.items():
    model = build_model(hidden_units=arch, dropout_rate=0.2, learning_rate=1

    history_tmp = model.fit(
        X_train_scaled, y_train,
        validation_data=(X_val_scaled, y_val),
        epochs=200,
        batch_size=32,
        callbacks=[early_stop, reduce_lr],
        verbose=0
    )

    pred_tmp = model.predict(X_test_scaled, verbose=0).flatten()
    rmse_tmp = np.sqrt(mean_squared_error(y_test, pred_tmp))
    mae_tmp = mean_absolute_error(y_test, pred_tmp)
    r2_tmp = r2_score(y_test, pred_tmp)

    comparison_results.append({
        "model": name,
        "architecture": arch,
        "rmse": rmse_tmp,
        "mae": mae_tmp,
        "r2": r2_tmp
    })

comparison_df = pd.DataFrame(comparison_results)
print(comparison_df)

```

Epoch 165: ReduceLR0nPlateau reducing learning rate to 0.0005000000237487257.

Epoch 175: ReduceLR0nPlateau reducing learning rate to 0.0002500000118743628.

Epoch 193: ReduceLR0nPlateau reducing learning rate to 0.0001250000059371814.

Epoch 52: ReduceLR0nPlateau reducing learning rate to 0.0005000000237487257.

Epoch 64: ReduceLR0nPlateau reducing learning rate to 0.0002500000118743628.

Epoch 74: ReduceLR0nPlateau reducing learning rate to 0.0001250000059371814.

Epoch 85: ReduceLR0nPlateau reducing learning rate to 6.25000029685907e-05.

Epoch 95: ReduceLR0nPlateau reducing learning rate to 3.125000148429535e-05.

Epoch 105: ReduceLR0nPlateau reducing learning rate to 1.5625000742147677e-05.

Epoch 48: ReduceLR0nPlateau reducing learning rate to 0.0005000000237487257.

Epoch 60: ReduceLR0nPlateau reducing learning rate to 0.0002500000118743628.

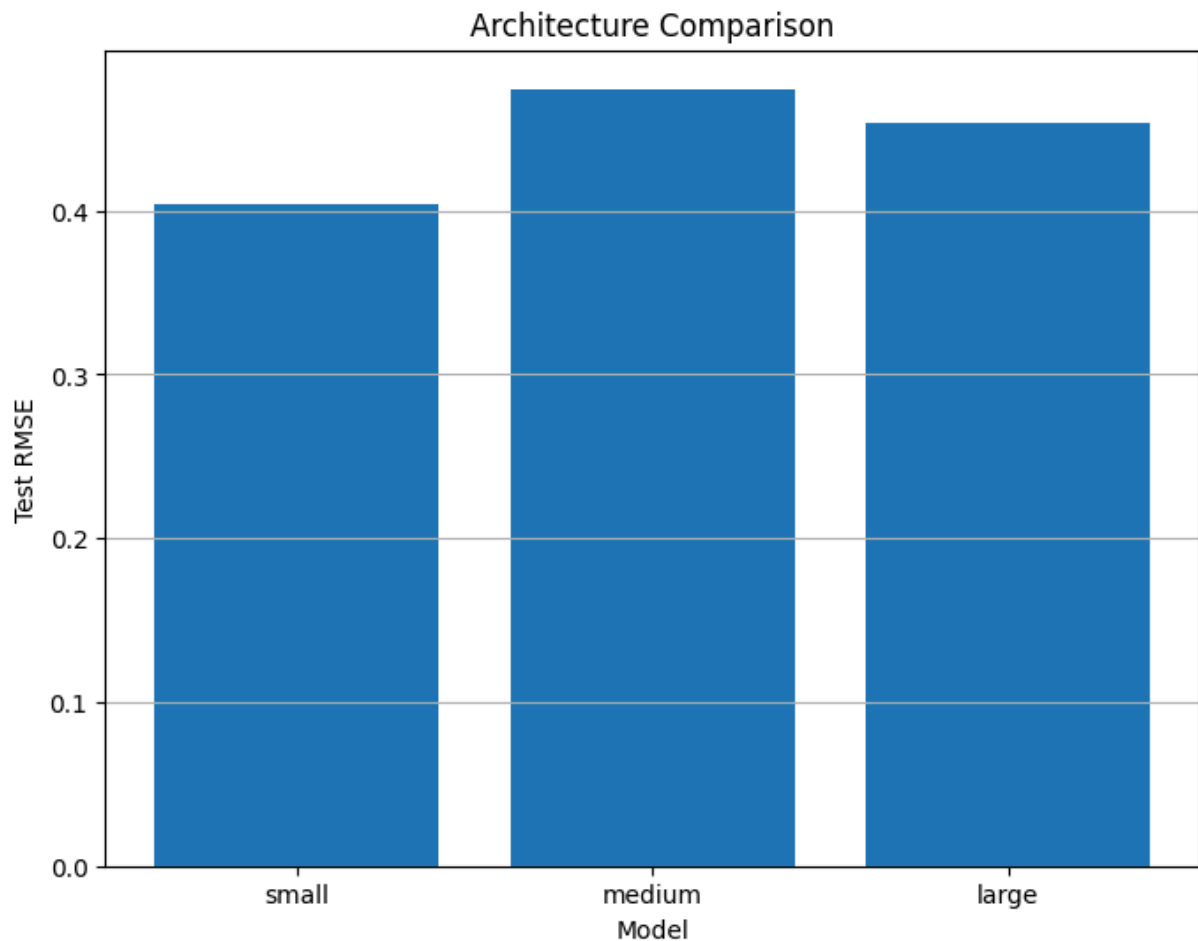
Epoch 84: ReduceLR0nPlateau reducing learning rate to 0.0001250000059371814.

Epoch 94: ReduceLR0nPlateau reducing learning rate to 6.25000029685907e-05.

Epoch 104: ReduceLR0nPlateau reducing learning rate to 3.125000148429535e-05.

	model	architecture	rmse	mae	r2
0	small	[32, 16]	0.404338	0.260092	0.985923
1	medium	[128, 64, 32]	0.474375	0.315828	0.980624
2	large	[256, 128, 64, 32]	0.453961	0.299399	0.982256

```
In [60]: plt.figure(figsize=(8, 6))
plt.bar(comparison_df["model"], comparison_df["rmse"])
plt.xlabel("Model")
plt.ylabel("Test RMSE")
plt.title("Architecture Comparison")
plt.grid(axis="y")
plt.savefig('plots/architecture_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```

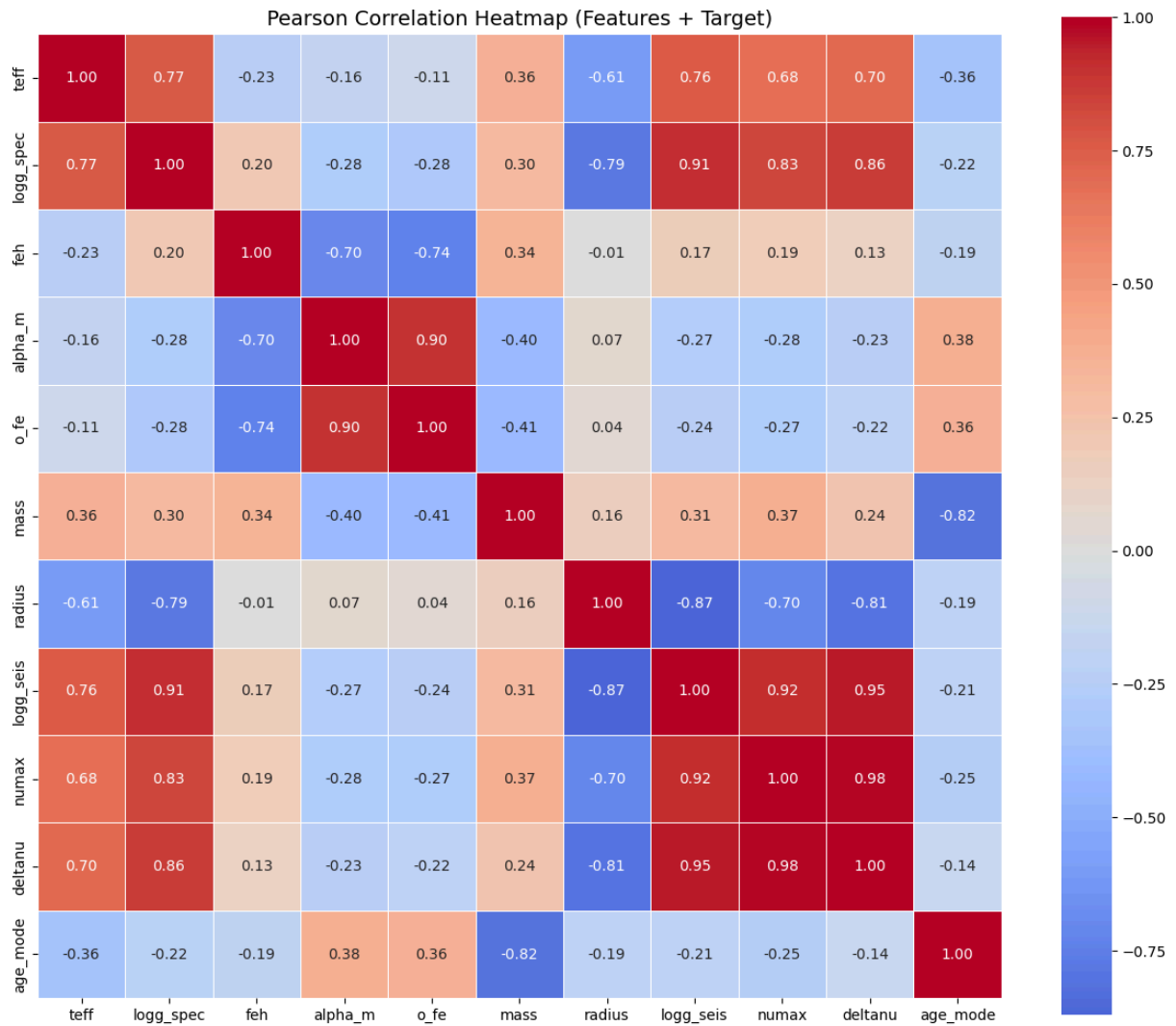


Additional Diagnostic Visualisations

```
In [61]: # 1. Correlation Heatmap of Input Features + Target
import seaborn as sns

corr_cols = feature_cols + [target_col]
corr_matrix = data[corr_cols].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(
    corr_matrix,
    annot=True,
    fmt='.2f',
    cmap='coolwarm',
    center=0,
    square=True,
    linewidths=0.5
)
plt.title('Pearson Correlation Heatmap (Features + Target)', fontsize=14)
plt.tight_layout()
plt.savefig('plots/pearson_correlation_heatmap_features_target.png', dpi=150)
plt.show()
```

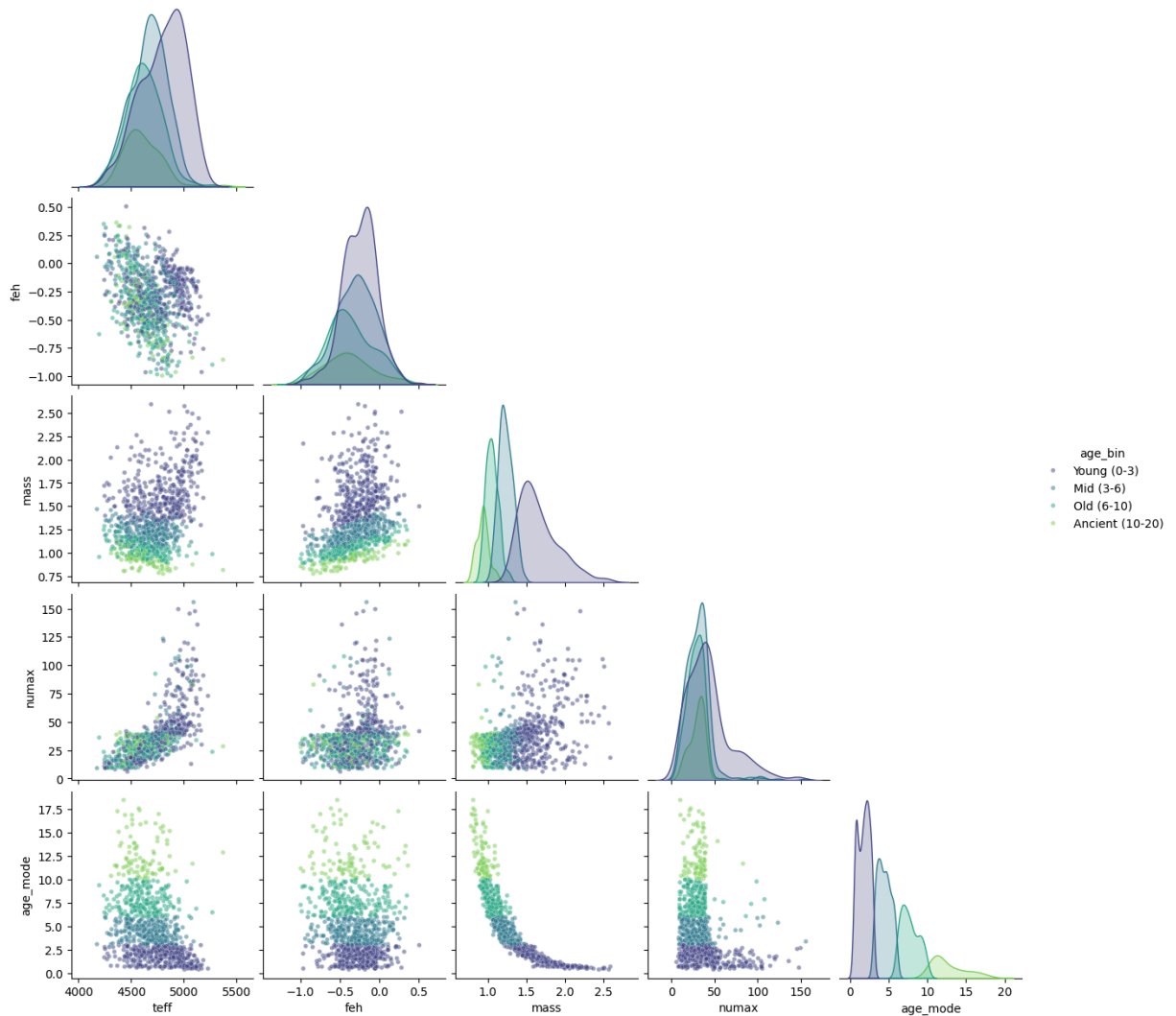


```
In [62]: # 2. Pair Plot of Selected Key Features coloured by Age
pair_cols = ['teff', 'feh', 'mass', 'numax', 'age_mode']
pair_data = data[pair_cols].copy()

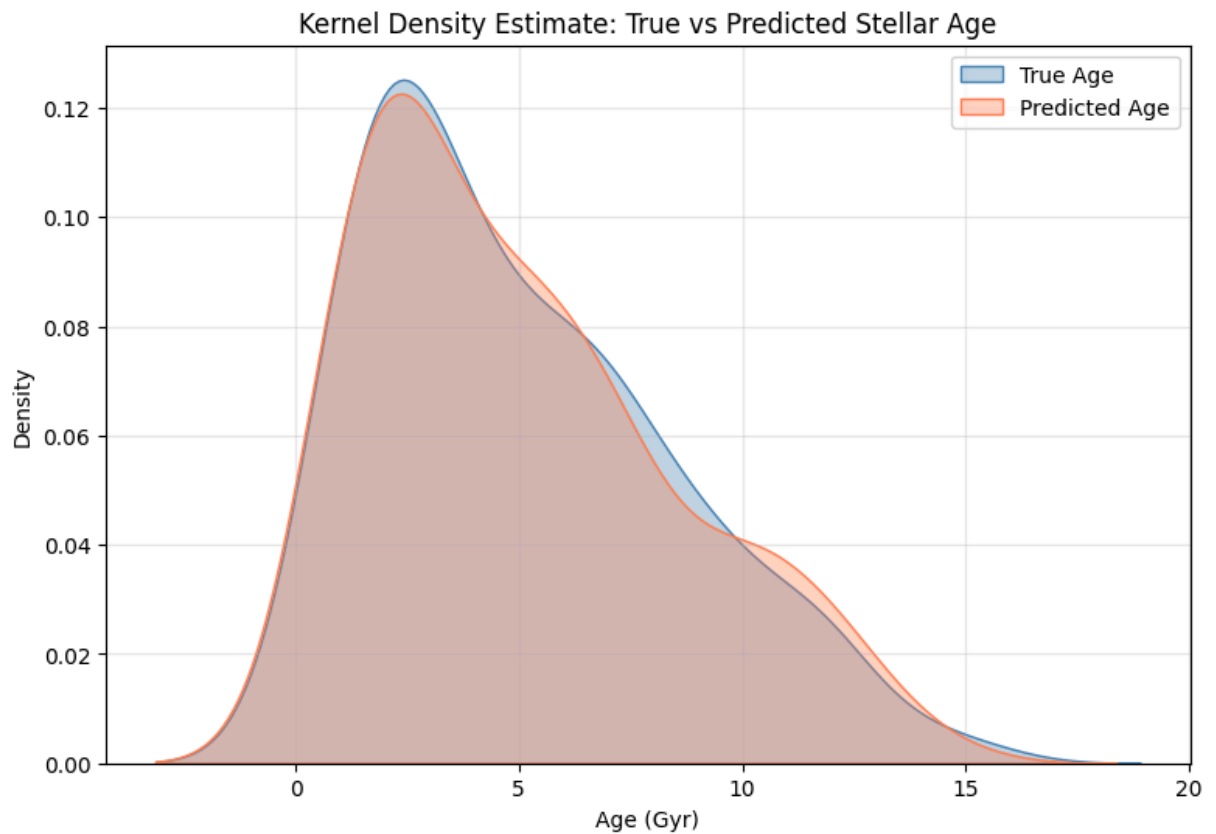
# Bin age into categories for colouring
pair_data['age_bin'] = pd.cut(
    pair_data['age_mode'],
    bins=[0, 3, 6, 10, 20],
    labels=['Young (0-3)', 'Mid (3-6)', 'Old (6-10)', 'Ancient (10-20)']
)

g = sns.pairplot(
    pair_data,
    hue='age_bin',
    palette='viridis',
    diag_kind='kde',
    plot_kws={'alpha': 0.5, 's': 15},
    corner=True
)
g.figure.suptitle('Pair Plot of Key Features (coloured by Age Bin)', y=1.02,
plt.savefig('plots/pair_plot_of_key_features_coloured_by_age_bin.png', dpi=1
plt.show()
```

Pair Plot of Key Features (coloured by Age Bin)

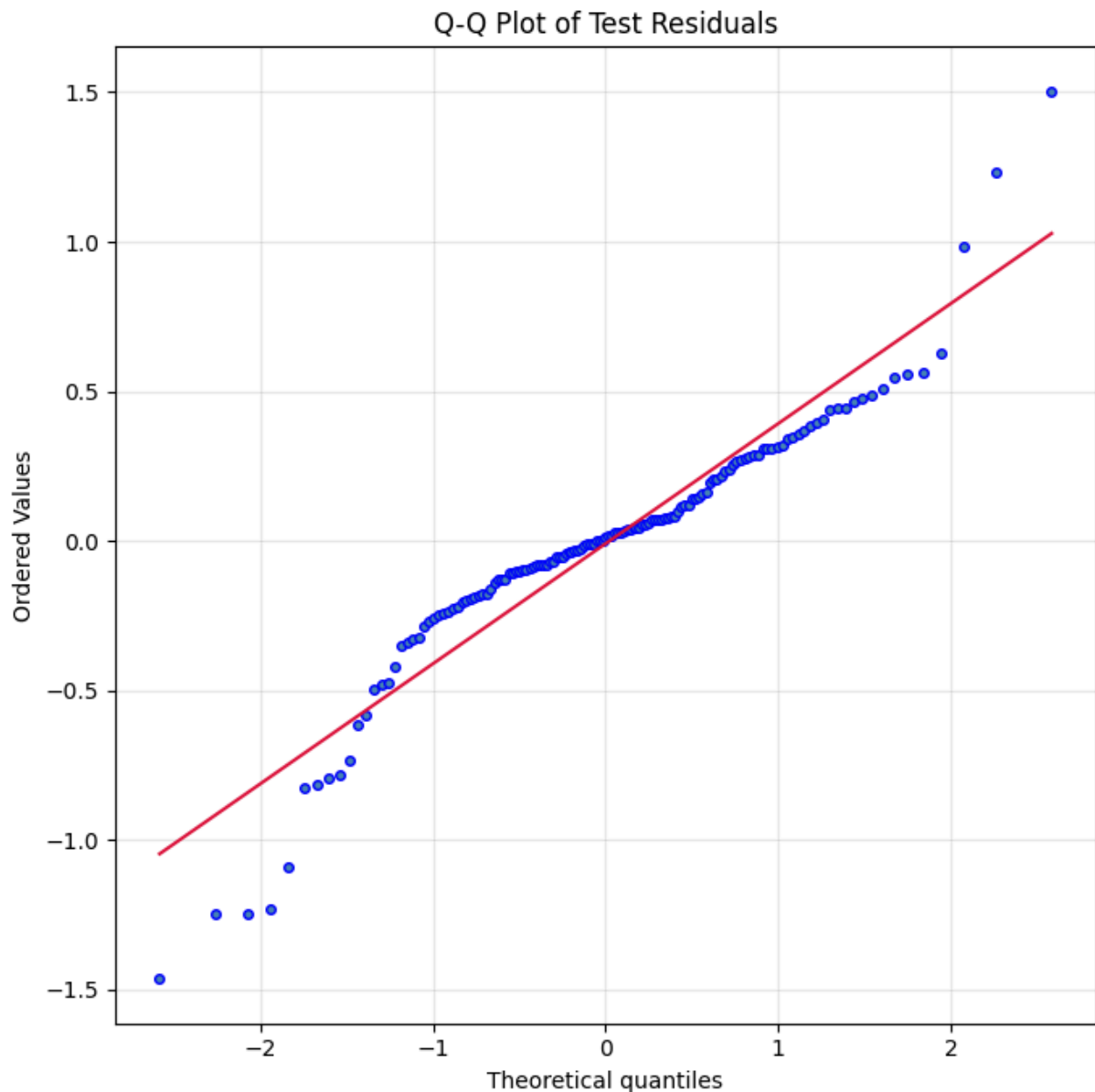


```
In [63]: # 3. KDE Overlay: True vs Predicted Age Distribution
plt.figure(figsize=(9, 6))
sns.kdeplot(y_test, label='True Age', fill=True, alpha=0.35, color='steelblue')
sns.kdeplot(y_pred, label='Predicted Age', fill=True, alpha=0.35, color='coral')
plt.xlabel('Age (Gyr)')
plt.ylabel('Density')
plt.title('Kernel Density Estimate: True vs Predicted Stellar Age')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('plots/kernel_density_estimate_true_vs_predicted_stellar_age.png')
plt.show()
```



```
In [64]: # 4. Q-Q Plot of Residuals (check for normality)
from scipy import stats

fig, ax = plt.subplots(figsize=(7, 7))
stats.probplot(residuals.values, dist='norm', plot=ax)
ax.set_title('Q-Q Plot of Test Residuals')
ax.get_lines()[0].set_markerfacecolor('steelblue')
ax.get_lines()[0].set_markersize(4)
ax.get_lines()[1].set_color('crimson')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('plots/q_q_plot_of_test_residuals.png', dpi=150, bbox_inches='tight')
plt.show()
```



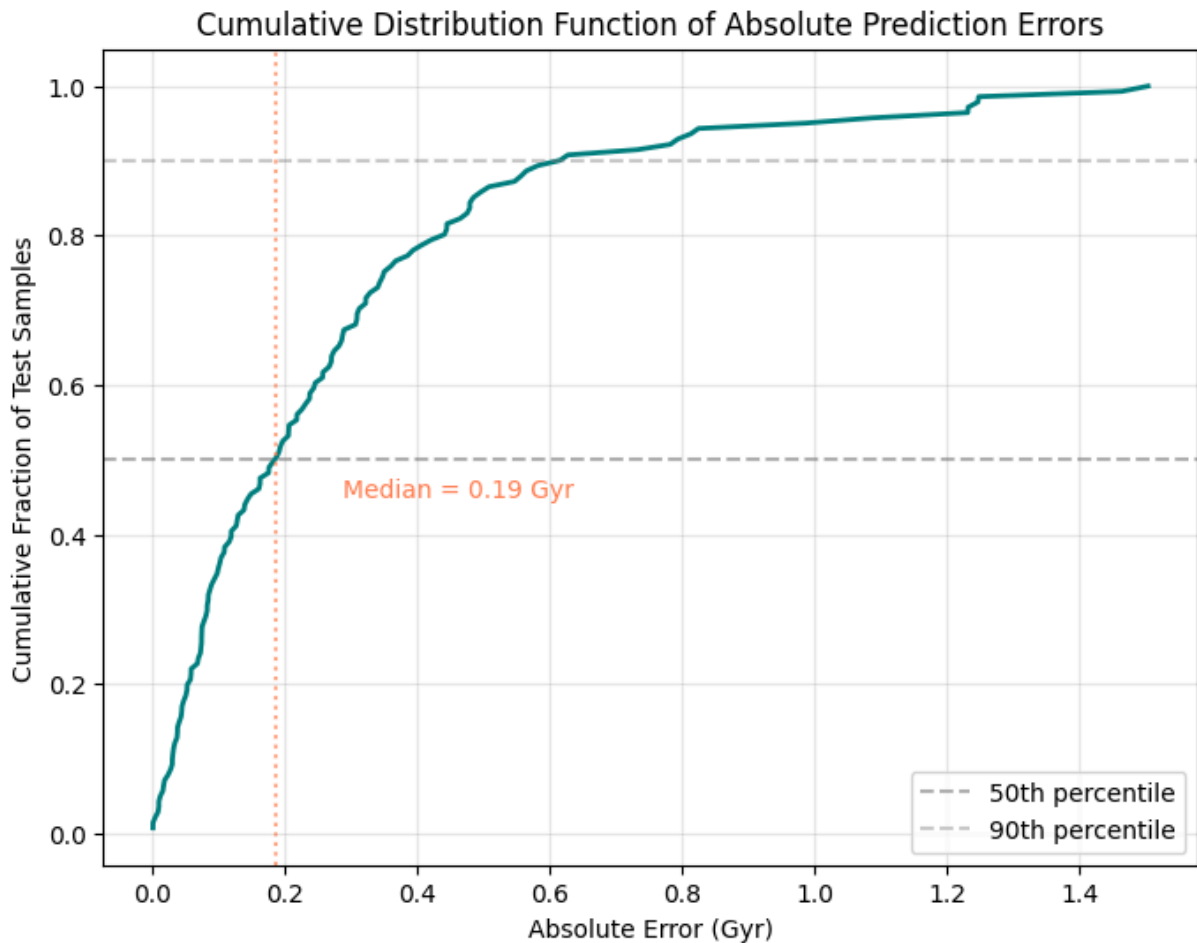
```
In [65]: # 5. Cumulative Distribution of Absolute Errors
abs_errors = np.abs(y_test.values - y_pred)
sorted_err = np.sort(abs_errors)
cdf = np.arange(1, len(sorted_err) + 1) / len(sorted_err)

plt.figure(figsize=(8, 6))
plt.plot(sorted_err, cdf, linewidth=2, color='teal')
plt.axhline(0.5, linestyle='--', color='grey', alpha=0.6, label='50th percer')
plt.axhline(0.9, linestyle='--', color='grey', alpha=0.4, label='90th percer')

# Annotate the median error
median_err = np.median(abs_errors)
plt.axvline(median_err, linestyle=':', color='coral', alpha=0.7)
plt.text(median_err + 0.1, 0.45, f'Median = {median_err:.2f} Gyr', fontsize=12)

plt.xlabel('Absolute Error (Gyr)')
plt.ylabel('Cumulative Fraction of Test Samples')
plt.title('Cumulative Distribution Function of Absolute Prediction Errors')
plt.legend()
```

```
plt.grid(True, alpha=0.3)
plt.savefig('plots/cumulative_distribution_function_of_absolute_prediction_e
plt.show()
```



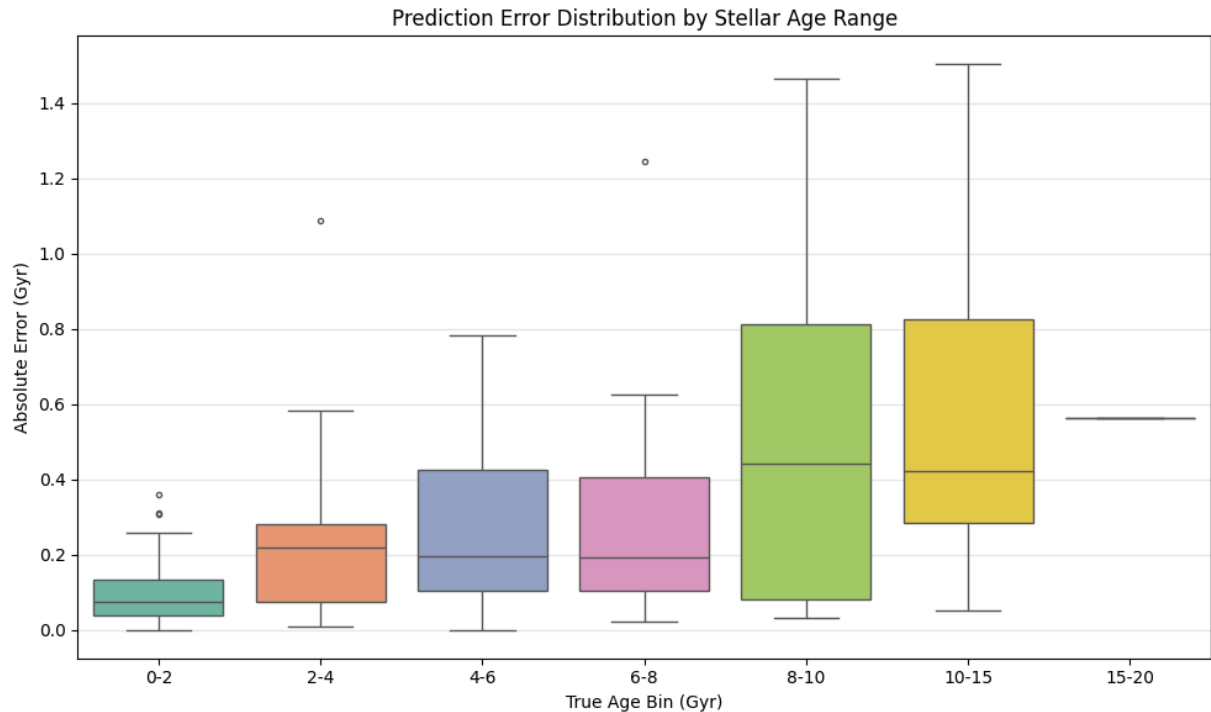
```
In [66]: # 6. Box Plot of Absolute Error grouped by True-Age Bins
test_results_tmp = pd.DataFrame({
    'true_age': y_test.values,
    'abs_error': np.abs(y_test.values - y_pred)
})
test_results_tmp['age_bin'] = pd.cut(
    test_results_tmp['true_age'],
    bins=[0, 2, 4, 6, 8, 10, 15, 20],
    labels=['0-2', '2-4', '4-6', '6-8', '8-10', '10-15', '15-20']
)

plt.figure(figsize=(10, 6))
sns.boxplot(x='age_bin', y='abs_error', data=test_results_tmp,
            palette='Set2', fliersize=3)
plt.xlabel('True Age Bin (Gyr)')
plt.ylabel('Absolute Error (Gyr)')
plt.title('Prediction Error Distribution by Stellar Age Range')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('plots/prediction_error_distribution_by_stellar_age_range.png',
plt.show()
```

```
/var/folders/1j/6168x29956dgb6w5h22437l00000gn/T/ipykernel_31378/2415867960.  
py:13: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(x='age_bin', y='abs_error', data=test_results_tmp,
```



```
In [67]: # 7. Permutation Feature Importance
from sklearn.inspection import permutation_importance

perm_result = permutation_importance(
    baseline_model, X_test_scaled, y_test,
    n_repeats=15, random_state=42,
    scoring='neg_mean_squared_error'
)

sorted_idx = perm_result.importances_mean.argsort()

plt.figure(figsize=(10, 7))
plt.barh(
    [feature_cols[i] for i in sorted_idx],
    perm_result.importances_mean[sorted_idx],
    xerr=perm_result.importances_std[sorted_idx],
    color='teal', alpha=0.8
)

plt.xlabel('Mean Decrease in Neg-MSE')
plt.title('Permutation Feature Importance (Baseline Model)')
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.savefig('plots/permutation_feature_importance_baseline_model.png', dpi=100)
plt.show()
```

[illegible]

[illegible]

5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	2ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step
5/5		0s	1ms/step

